Author .

ee99ce3e-38b3-493a-b0f1-a7c03a5f36be_doc_Thesis_Report_...

CANTIKA RENATA ZESTY

Document Details

Submission ID
trn:oid:::3618:137445876

Submission Date
May 2, 2026, 11:17 PM GMT+5:30

Download Date
May 2, 2026, 11:20 PM GMT+5:30

File Name
ee99ce3e-38b3-493a-b0f1-a7c03a5f36be_doc_Thesis_Report_v2.docx

File Size
721.6 KB

86 Pages

22,367 Words

125,610 Characters

7% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography

Match Groups

79 Not Cited or Quoted 4%
Matches with neither in-text citation nor quotation marks

61 Missing Quotations 3%
Matches that are still very similar to source material

0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

5% Internet sources
3% Publications
5% Submitted works (Student Papers)

Match Groups

- **79** Not Cited or Quoted 4%
  Matches with neither in-text citation nor quotation marks
- **61** Missing Quotations 3%
  Matches that are still very similar to source material
- **0** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

5% Internet sources
3% Publications
5% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Internet	dspace.iiuc.ac.bd 1%
2	Internet	arxiv.org <1%
3	Internet	mihaicosma.com <1%
4	Internet	superml.dev <1%
5	Internet	gwern.net <1%
6	Internet	haolibai.github.io <1%
7	Student papers	Western Balkans University on 2026-02-25 <1%
8	Internet	minjiazhang.github.io <1%
9	Internet	synthical.com <1%
10	Student papers	University of Greenwich on 2025-04-21 <1%

11 Internet
cmu-l3.github.io <1%

12 Publication
Thimira Amaratunga. "Understanding Large Language Models", Springer Science ... <1%

13 Internet
gwern.obormot.net <1%

14 Internet
core.ac.uk <1%

15 Internet
medium.com <1%

16 Internet
www.techrxiv.org <1%

17 Student papers
University of Sheffield on 2026-04-29 <1%

18 Internet
aclanthology.org <1%

19 Internet
www.gwern.net <1%

20 Internet
123dok.net <1%

21 Student papers
California State University, Fresno on 2024-03-14 <1%

22 Student papers
Higher Education Commission Pakistan on 2017-07-14 <1%

23 Internet
gsarti.com <1%

24 Internet
nucl-ph.chinaxiv.org <1%

25 Student papers
Columbia University on 2025-12-19 <1%

26 Student papers
University of Hong Kong on 2025-10-31 <1%

27 Publication
Juntao Hu, You Yang, Yongzhi An, Lu Yao. "Dual-Spatial Normalized Transformer f... <1%

28 Student papers
London Metropolitan University on 2017-01-23 <1%

29 Student papers
University of Edinburgh on 2025-04-14 <1%

30 Student papers
University of Hong Kong on 2025-08-02 <1%

31 Student papers
University of Northampton on 2023-06-25 <1%

32 Internet
deepai.org <1%

33 Internet
diposit.ub.edu <1%

34 Internet
theonly1.tistory.com <1%

35 Publication
Andreas S. Weigend. "Time series analysis and prediction using gated experts wit... <1%

36 Student papers
BPP College of Professional Studies Limited on 2023-01-09 <1%

37 Student papers
Institute of Technology, Tralee on 2018-04-03 <1%

38 Student papers
University of Greenwich on 2026-04-22 <1%

39 Internet
curve.carleton.ca <1%

40 Internet
johal.in <1%

41 Internet
tnq177.github.io <1%

42 Internet
unsworks.unsw.edu.au <1%

43 Internet
www.frontiersin.org <1%

44 Internet
www.mdpi.com <1%

45 Publication
"Computational Intelligence in Engineering Science", Springer Science and Busin... <1%

46 Publication
Kalim Sattar, Qasim Umer, Dinara G. Vasbieva, Sungwook Chung, Zohaib Latif, Ch... <1%

47 Student papers
Liverpool John Moores University on 2023-06-05 <1%

48 Student papers
Queen's University of Belfast on 2020-12-10 <1%

49 Student papers
The French - Vietnamese Center for Management Education on 2026-04-21 <1%

50 Student papers
The University of Texas at San Antonio on 2026-04-28 <1%

51 Student papers
University of Central Florida on 2024-04-18 <1%

52 Student papers
University of Edinburgh on 2021-01-22 <1%

53 Student papers
Universität St. Gallen on 2026-04-16 <1%

54 Internet
ar5iv.labs.arxiv.org <1%

55 Internet
edoc.ub.uni-muenchen.de <1%

56 Internet
huggingface.co <1%

57 Internet
irlab.science.uva.nl <1%

58 Internet
papers.neurips.cc <1%

59 Internet
unixer.de <1%

60 Publication
"China Conference on Knowledge Graph and Semantic Computing and Internatio... <1%

61 Publication
"ECAI 2020", IOS Press, 2020 <1%

62 Publication
"Machine Learning and Knowledge Discovery in Databases. Research Track", Spri... <1%

63 Publication
Chen, Zixiang. "On the Theory of Deep Learning and the Alignment of Large Lang... <1%

64 Student papers
De Montfort University on 2022-12-22 <1%

65 Publication
Eilif Hjelseth, Sujesh F. Sujan, Raimar J. Scherer. "ECPPM 2022 – eWork and eBusin... <1%

66 Student papers
Imperial College of Science, Technology and Medicine on 2025-09-09 <1%

67 Student papers
Instituto Politécnico de Coimbra on 2026-02-02 <1%

68 Publication
Jiacheng Liu, Peng Tang, Wenfeng Wang, Yuhang Ren, Xiaofeng Hou, Pheng Ann ... <1%

69 Publication
Khandelwal, Urvashi. "Improving Neural Language Models with Black-Box Analys... <1%

70 Publication
Park, Core Francisco. "Deep Learning as a Scientific Tool and a Model Organism o... <1%

71 Student papers
Queen Mary and Westfield College on 2022-08-15 <1%

72 Publication
Sandhya Arora, Urmila Shrawankar, Maya Ingle, Indu Bhagat. "Computing Techn... <1%

73 Publication
Shaier, Sagi. "Factual Knowledge-Enhanced Question Answering in Dynamic Envir... <1%

74 Student papers
The NorthCap University, Gurugram on 2023-11-27 <1%

75 Student papers
Vellore Institute of Technology on 2026-04-26 <1%

76 Publication
Zihui Yang, Yihang Sun, Heng Zhang, Chong Chen, Ziwen Mo, Zhixing Gu, Taoshe... <1%

77 Internet
dokumen.pub <1%

78 Internet
flore.unifi.it <1%

79 Internet
github.com <1%

80 Internet
jmlr.csail.mit.edu <1%

81 Internet
lirias.kuleuven.be <1%

82 Internet
odr.chalmers.se <1%

83 Internet
repo.lib.duth.gr <1%

84 Internet
vdoc.pub <1%

85 Internet
videolectures.net <1%

86 Student papers
Carnegie Mellon University on 2024-12-06 <1%

87 Publication
Duke, Brendan. "Sparse Attention Mechanisms in Vision Transformers: From Spat... <1%

88 Student papers
Liverpool John Moores University on 2024-04-04 <1%

89 Publication
Paolo Ferro, Harinadh Vemanaboina, Chander Prakash. "Computational Techniqu... <1%

90 Publication
Renz-Wieland, Florian Alexander. "Adaptive Parameter Servers", Technische Univ... <1%

91 Student papers
University of Edinburgh on 2024-08-16 <1%

92 Publication
"Computer Vision – ECCV 2022 Workshops", Springer Science and Business Media ... <1%

93 Student papers
Imperial College of Science, Technology and Medicine on 2025-09-09 <1%

94 Student papers
Letterkenny Institute of Technology on 2024-08-30 <1%

95 Publication

Nicholas Kluge Corrêa, Sophia Falk, Shiza Fatimah, Aniket Sen, Nythamar de Olive... <1%

96 Publication

Patel, Pratyush. "Structural Insights for LLM Serving Efficiency.", University of Wa... <1%

97 Student papers

UNICAF on 2024-07-01 <1%

98 Publication

Yingchao Zhang, Cheng Liu. "Basic concepts", Elsevier BV, 2026 <1%

99 Internet

openaccess.thecvf.com <1%

INTERNATIONAL ISLAMIC UNIVERSITY CHITTAGONG

CacheFlow: Memory-Aware Expert Scheduling for Efficient Mixture-of-Experts Models

Supervised by:

Jamil As Ad
Lecturer
Department of Computer Science and Engineering
International Islamic University Chittagong

Submitted By:

Mir Md. Tarhimul Quader C221017
Md. Iftaker Ahamed Sayem C221020
Turja Dutta C221026

BACHELOR OF SCIENCE IN COMPUTER SCIENCE AND ENGINEERING

Department of Computer Science and Engineering
International Islamic University Chittagong

May, 2026

Declaration

We hereby affirm the following statements regarding our thesis:

1. The thesis has been successfully completed as part of our undergraduate degree program at International Islamic University Chittagong.
2. The thesis work does not contain any previously published or third-party content without proper citation.
3. The thesis work has not been previously submitted for any other degree or diploma at any other university or institution.
4. We have appropriately acknowledged all significant sources of contribution in the thesis.

Student's Full Name and Metric ID:

Mir Md. Tarhimul Quader (C221017)

Md. Iftaker Ahamed Sayem (C221020)

Turja Dutta (C221026)

1

Supervisor's Declaration

I formally state that I have examined this thesis and claim it to be of sufficient quality and scope to be granted the undergraduate degree of Bachelor of Science in Computer Science and Engineering.

.....

Jamil As-Ad
Lecturer
Department of Computer Science and Engineering
International Islamic University Chittagong

Dedication

First and foremost, we thank the Almighty Allah for his generosity, which enabled us to finish our thesis despite a number of obstacles.

Second, we would like to thank Jamil As-Ad Sir, our supervisor, for his constant assistance and direction from the beginning of our study.

Acknowledgment

First of all, we are grateful to the Almighty Allah for his kindness, which allowed us to complete our thesis in spite of several challenges.

Second, we would like to express our appreciation to our supervisor **Jamil As Ad** Sir for his unwavering support and guidance from the start of our research.

Ethical Statement

Hereby we state that None of the unethical practices were used in the completion of our thesis work. The data we used for research purposes are original. We carefully checked every citation we used here. The writers of the work accept all the liabilities for any kind of violation of the thesis rule.

Abstract

Mixture-of-Experts (MoE) scale models have already shown impressive run-time performance on both natural language processing and multimodal problems, but their application is crippled by the bottleneck of GPU memory. Although the MoE architectures can efficiently compute with sparse expert activation, their implementation demands that all expert weight matrices simultaneously occupy the VRAM of consumer-grade and edge hardware (a cost that is prohibitive at state-of-the-art scales (20-30 GB) and impractical with consumer-grade and edge hardware). Current methods either suffer out of memory failures (DeepSpeed MoE on limited hardware) or impose an abhorrently high latency penalty (MoE-Infinity: 920 ms/token, 5.4 GB pinned host memory allocation). The current thesis proposes CacheFlow, a full three-level memory hierarchy that uncouples the demands of the GPU memory usage with the overall number of expert parameters, allowing effective MoE inference on resource-constrained systems without changing model specification or routing. The main innovation chains disk-resistive expert loads are pin-staged in CPU memory to the GPU VRAM slots via non-blocking asynchronous PCIe transfer. The system uses direct disk I/O through slicing safetensors (MmapExpertStore), intelligent pinned memory buffer management (nextstaging()) and LRU cache with intra-batch freezing (ExpertCache) to avoid the unintentional eviction of needed experts during the process of processing a batch of experts. Empirical analysis shows that CacheFlow can achieve 5- 9x peak memory reductions of GPU systems relative to baseline systems in limited capacity configuration. In particular, at capacity constraint C=1 (one expert slot per MoE layer) the system supports a maximum VRAM footprint of 988.17 MB and a per-token latency of 598.23 ms, but supports a 23-percent lower per-token latency of 462.27 ms/token at C=8 (1845.26 MB VRAM) Intra-batch freezing mechanism imposes a very small overhead (approximately 0.1% of latency) and ensures the computational accuracy. Enabling the execution of state-of-the-art MoE models on consumer-grade GPUs (RTX 3060, Jetson Orin with 12 GB VRAM) that hitherto needed specialized infrastructure, CacheFlow democratizes access to sparse architectures of the expert models and provides a viable implementation base over resource-constrained settings.

Keywords: Asynchronous expert prefetching, Edge device deployment, LRU cache with intra-batch freezing, Memory-efficient GPU scheduling, Mixture-of-Experts (MoE) inference, Resource-constrained inference optimization, Three-tier memory hierarchy.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Background and Motivation

With the development of Mixture-of-Experts (MoE) architectures, a new phase in large-scale deep learning has been reached, making it possible to train and deploy the most useful and capable language models with a set computational budget. Notable ones are Mistral 7B (scarcity of experts switching with 32 experts), qwen3-MoE-A2.7B and qwen3-MoE-57B (64 experts per layer) and Mixtral 8x7B (eight expert pathways per token). Their performance on many benchmarks is of state-of-the-art due to the ability of these models to send each token to a small number of specialized experts and achieve a greater model capacity without compromising efficient computation.

The theoretical merits of MoE models are well-developed: they offer good compute to parameter ratios over dense. A 60-expert MoE layer can process tokens by only passing them through 2-4 experts of choice, with compute cost of $O(\text{num\_selected\_experts})$ and parameters of $O(\text{num\_total\_experts})$. But the abundance of this parameter poses an acute practical difficulty: the weights of all the experts should be available when inferring, which require astronomical amounts of GPU VRAM. With 60 experts per layer, most of whom weigh approximately 100 MB, a typical MoE layer will necessitate 6 GB of expert parameters by itself. Together with transformer static parameters (embeddings, attention layers, output projections), KV caches of autoregressive decoding and activation buffer intermediates, can hotly demand GPU memory in the cumulative sum of 20-30 GB making it far beyond the limits of consumer-grade GPUs (typically 6-12 GB) and even many mid-range server accelerators.

This demonstrates VRAM bottleneck becoming one of the main limitations to the wide adoption of MoE models. Although the initial training and inference investigations by the MoE frequently accounted for the access to high specialized apparatus with a vast amount of recollection, these domains do not regularly incorporate them in practice. The constraints of academic researchers, smaller organizations and edge

computing applications necessitate either giving up MoE models altogether or making crude approximations (expert pruning, quantization) that reduce model quality. It inspires the design of effective memory-conscious inference that can isolate the exposures to the lifetime memory of the graphics card and the overall number of the experts and realize full-capacity MoE on resource-limited devices.

1.2 Problem Statement

The modalities that have been developed to overcome the bottleneck presented by the MoE memory can be divided into two main categories that pose considerable limitations.

Distributed Expert Parallelism: Systems like DeepSpeed-MoE and Megatron use expert parallelism, where experts are distributed to multiple GPUs, with coordinated communication. Whereas it can be used successfully in datacenter settings with high-bandwidth interconnections, this method adds considerable complexity and is otherwise closed off to practitioners using single-GPUs or low hardware cost. Moreover, synchronous communication across expert boundaries establishes hard latency floors which makes it inapplicable to latency-sensitive applications.

Host Memory Offloading (Naive Staging): Another approach is to offload the weights of an expert loads host (CPU) pinned RAM before inference, then loads it to the GPU when needed. This has the advantage (as seen with MoE-Infinity and other disk-offloading mechanisms) of decreasing the VRAM requirement by the GPU, but has the drawback of placing a new bottleneck of bottleneck the pre-fill phase itself as sequential. The forward pass is blocked until the PCIe transfer finishes when the system needs to transfer an expert out of the pinned host memory to the GPU memory and continues to calculate on it. This empirically yields a 9-20 ms/token or greater latency penalty, a 1.5-3x penalty versus the baseline systems. In addition, keeping all experts in pinned host RAM, requires 5-10 GB of host memory allocations, limiting the available GPU capacity and the number of requests in parallel inference.

Joint Limitations: Both solutions provide the opportunity to miss a crucial opportunity: overlapping disk I/O, PCIe transfers and computation of the GPU asynchronously. The critical path of a synchronous design is: (disk read) → (wait to completion) → (PCIe transfer) → (wait to completion) → (GPU compute). All the stages are serial and introduce dire latency punishments. Moreover, both systems have no principled caching policy to leverage locality of time in access patterns of experts; in fact experts get moved repeatedly when accessed in succession.

The fundamental reason of these constraints is architectural: the current systems lack the realization of the real three-tier memory hierarchy with non-blocking asynchronous operations. They can either consolidate, or implement synchronous staging

(latency failure); they can consolidate experts in GPU memory (scalability failure), or consolidate experts in pinned host memory (latency failure). There is no known framework that smoothly connects disk I/O with pinned staging to asynchronous GPU transfer having a smart caching framework.

1.3 Proposed Solution: CacheFlow

To overcome these restrictions, CacheFlow uses a fully developed three tiered hierarchy of memory and asynchronous memory execution model. It is designed in the following way:

- **Tier 1: Disk Storage (Persistent Archive):** The expert weights are stored in disk format in safetensors format - a compact binary-based encoding that allows them to be sliced randomly. As opposed to naive methods that read complete weight files, CacheFlow only reads a slice of the expert that is requested, often 100-256 MB per expert which slows down the I/O tally considerably.
- **Tier 2: Pinned CPU Memory (Staging Buffers):** A tiny fixed sized pool of pinned (page-locked) CPU buffers is a stage where weights are gathered during disk loading. The most important novelty is that the number of staging buffers (4-8) is independent of the number of experts (60). This enables the system to pipe several disk reads and PCIe transfers simultaneously, taking advantage of the high I/O throughput of the current storage devices and PCIe interconnects.
- **Tier 3: GPU VRAM (Working Set Cache):** This is a fixed capacity LRU based on GPU memory, which contains the sub-group of experts that are being utilized. More importantly, this cache uses a freezing system of intra-batches: before every forward pass, all the specialists (identified by the routing of tokens) are freeze-locked in the cache, ensuring that they will not be evicted during the computation of the batch. This removes the risks of corruption and allows loading cold experts safely asynchronously.

Asynchronous Execution Model: The system concurrently schedules disk I/O and PCIe transfers with GPU computation, through torch.cuda.Stream. As the CPU reads Expert A weights off disk, Expert B weights (already loaded into VRAM) in PCIe are being loaded into the GPU and Expert C weights (already in cache) in the CPU are being calculated. Effective latency is drastically lowered by this triple concurrency.

Transparent to Application: CacheFlow leaves the underlying MoE architecture, model structure and routing mechanisms unchanged. It is a drop-in replacement

of conventional parallel expert implementations and is widely compatible with existing frameworks and models should an expert be being implemented in parallel.

1.4 Research Objectives

This thesis had the following objectives:

- **Goal 1:** Develop and provide a low-memory three-tier architecture of a MoE inference system that does not connect the VRAM needed in GPUs with the number of potential expert parameters, as well as with computational accuracy.
- **Objective 2:** Show that asynchronous pre-fetching through non-blocking PCIe streams can be useful in overlapping disk I/O, data staging and GPU computation to alleviate the latency discharge that disk offloading normally incurs.
- **Objective 3:** Empirically test the system under varied capacity limits (GPU memory budgets of 900 MB to 2 GB) and system designs (Granite-3B, qwen3-MoE variants) measuring the extraction of the memory-latency trade-off frontier.
- **Objective 4:** Inform architectural knowledge and design considerations helping practitioners (and others) to implement large MoE models on resource-limited hardware, such as memory-optimal capacity plans and staging buffer tuning plans.

1.5 Key Contributions

The CacheFlow system and the assessment that goes along with it add the following new contributions to field:

Another Hierarchy of three-tier memory with asynchronous staging: CacheFlow presents an architectural design founded on three levels of concerns, which include a disk, pinned host RAM and GPU VRAM, with non-blocking since asynchronous non-blocking data transfer. Such a design is radically different as compared to previous work and satisfies several important properties: (a) disk I/O scales as the size of one expert, rather than the entire weight file; (b) pinned memory allocation can scale as the number of staging buffers (usually 4-8) and not the number of experts; (c) GPU cache capacity can be independently relaxed with the capacity constraint C without impacting on host-level allocations.

Correctness and Safety Intra-Batch Freezing: The freeze many mechanism is a simple but effective correctness guarantee: correctness guarantees of experts that

may be needed during a batch can never be evicted during the computation of that batch. This removes deadlocks, data races and loss of partial result which would otherwise happen in naive asynchronous systems. The freezing protocol incurs minimal overhead (approximately 0.1% latency) with transparent safety semantics.

Quantified Memory-Latency Trade-off Frontier: Experimental testing has shown CacheFlow attains 5-9× of peak GPU VRAM reductions relative to standard systems through limited capacity configurations. Specifically:

- $C = 1$ (ultra-constrained, a few experts slots per MoE layer): VRAM peak 988 MB, 598 ms/token latency.
- At $C = 8$ (moderate constraint, 8 expert slots per MoE layer): 1845 MB peak VRAM, 462 ms/token latency.
- The throughput was constant at 100-130 tokens/minute over all configurations, in contrast to almost zero throughput with constrained systems at baseline settings.

This is a sensible Pareto frontier at which practitioners have the freedom to select operating points trading off between memory savings and reasonable latency overheads.

Generalization of Architecture in Various MoE Models: The CacheFlow design is model-agnostic and scales appropriately with a variety of MoE architectures. The assessment of Granite-3B (8 experts, 48 MB at once), the primary test setup (60 experts, 107 MB at once) and the analysis of qwen3-MoE (64 experts, 256 MB at once) indicate that there are consistently good performance attributes. This system is demonstrated to be more effective with increased sizes of the experts, thus especially useful in next-generation large-scale MoE models.

Practical Deployment Enablement: CacheFlow largely makes production MoE models runable on consumer and mid-range GPUs by cutting gpu memory claims by orders of magnitude with respect to other midspecifications and with a realistic latency. This makes state-of-the-art model architectures accessible to all users and lowers the per-inference expenses in datacenter environments with better resource utilization.

1.6 Thesis Organization

The rest of this thesis follows in the following manner: **Chapter 2** gives a in-depth literature review covering the previous studies in MoE model architectures, memory-efficient inference methods and disk-offloading systems, putting the contribution of CacheFlow in context with other works on the topic. **Chapter 3** includes the full

methodological design of the CacheFlow system, with the three-level memory hierarchy, asynchronous execution model, the freezing mechanism and the mathematical modeling of memory budgets and capacity constraints. **Chapter 4** presents detailed experimentation, comparing CacheFlow with systems based on baseline systems (DeepSpeed MoE and MoE-Infinity), memory-latency trade-off analysis under the influence of different capacity constraints and cache hits and prefetching dynamics, as well as generalization to support a wide array of model architectures. The thesis ends with **Chapter 5**, which provides a synthesis of the principal results, discusses system restrictions and their implications as well as highlights the promising research directions in the future. At all times, with high respect to high quantitative assessment and clarity of presentation as well as applicability to deployment situations.

Chapter 2

Literature Review

2.1 Overview

The modern large-scale models of language have developed based on the breakthroughs in terms of architecture, optimization and systems design within the last three decades or so. Initial literature on Mixture-of-Experts (MoE) models proposed the use of conditional computation: a subsection of the model can specialize on specific areas of inputs. This was done with the goal of increasing the representational power of the network without corresponding increases in the cost of computation. Simultaneously, the development of optimization algorithms, normalization algorithms and attention systems made the deep neural networks effective in scaling. These advances led to the development of Transformer-based architectures which were soon the standard design of sequence modeling. The more complex the models became in terms of millions of parameters, then billions of parameters, then even of trillions of parameters, the greater the increase in performance became in direct proportion to the corresponding rise in both computation and memory requirements.

Transformer changed the face of natural language processing. It rendered training very parallelisable and offered effective long -range dependency modeling. This was later followed up by more research that enhanced expressiveness, stability and scalability. Further innovations included as better normalisation techniques, new activation functions, sparse attention systems and large scale distributed training systems. Although these advances increased the quality of models and the speed of training, they increased the level of inference to resource requirements. In dense ones, all of the parameters are required to be held in memory at inference, posing a difficulty in deploying the technology to a low-hardware device.

The response to the scaling problem was based on the sparse Mixture-of-Experts. MoE models incurred a massive number of parameters without proportionate raise in per-token compute by simply enabling a limited number of experts per-token.

However, as we have seen, inference systems do require all expert weights to be loaded on the GPU. As a result, the number of memory used tends to increase with the overall number of the experts, rather than the number of the experts at work. This restricts the capability of consumer GPUs to execute large MoE models.

Recent studies have changed the focus on inference time optimization and improved memory management. Efficient attention, key-value cache memory paging, task switching between the CPU and the GPU and distributed sharding are some of the techniques that are expected to minimize memory bottlenecks. Although useful, these approaches are usually operated at a coarse granularity, not scheduling context of a specific system and are oriented towards multi-gpu implementation instead of single-gpu environment. Therefore, there is still a research void of fine-grained, context-dependent expert scheduling on one single-processor graphics card.

In this chapter, a review of essential MoE literature, Transformer design, scaling techniques and current inference systems takes the structure of a paper-by-paper. Both works are analyzed in terms of their idea, technical contribution, implementation and empirical findings, as well as limitations, primarily in terms of inference time memory use. The history of the review: Mixture-of-Experts designs, the emergence of Transformers, large-scale sparse models and current serving and memory-management designs. The chapter also concludes with a comparative summary and a definite research gap that drives CacheFlow.

2.2 Foundational and Modern Architectures for Scalable Neural Models

This part presents systematic, paper-by-paper discussions of traditional Mixture-of-Experts, approach to optimization, Transformer structures, large scale language model and current inferential computations. The content of each subsection is a balanced paragraph, where one of the works presents its important idea, method, results and limitations, especially concerning the inference-time memory scalability.

2.2.1 Adaptive Mixtures of Local Experts by Jacobs et al. (1991)

Jacobs et al. (1991) portrayed Adaptive Mixtures of Local Experts. Instead, they constructed a neural network that had several expert networks, each specializing in a segment of the input space. The process of a gating network, which is dynamically trying to assign weight to the output of each expert, is based on the soft partitioning process of probabilistic nature. An approximation of functionality in experiments was

higher than approximate converging monolithic networks. Nonetheless, the work concentrated on small-scale systems and it did not take into account memory limitations of hardware. The entire expert parameters were assumed to be loaded on inference, which upon the scale was a reasonable assumption but became an issue.

2.2.2 Hierarchical Mixtures of Experts by Jordan and Jacobs (1994)

Jordan and Jacobs (1994) built on the Mixture-of-Experts framework to introduce hierarchical gating structures which were optimized using Expectation-Maximisation. Davids would now have the ability to specialise at various levels, which would result in a higher degree of flexibility and capacity. Compared to flat mixtures the hierarchical design was better at classification and regression. As in the previous work, it also assumed all the experts were in memory at runtime and it did not concern runtime memory efficiency. However, the hierarchical concept also forms the basis of the contemporary sparse Transformer-based designs, according to which experts are conditionally activated, but usually reside in memory.

2.2.3 Adam: A Method for Stochastic Optimization by Kingma and Ba (2014)

Kingma and Ba (2014) presented Adam, which is an optimizer that combines adaptive learning rates and momentum to accelerate training. Adam follows first and second-order moments of gradients, stabilizing updates in large-scale training. It was made a default option of deep Transformers and MoE models due to its strength and little tuning. Adam on the other hand is only able to touch on the training but has no influence on the inference-time usage of memory. It assisted scaling training, but failed to alleviate the parameter-resident bottleneck that has arisen in implementing those trained models.

2.2.4 Distilling the Knowledge in a Neural Network by Hinton et al. (2015)

The authors Hinton et al. (2015) presented a method of knowledge distillation which is a more compact student model that has the accuracy of a larger neural network. The process takes the softened output probabilities of the teacher and transfers it to the student causing a number of parameters to be reduced in order to attain competitive performance. It has been experimentally verified that distilled models can be as accurate as larger networks by re-training, but rather than managing memory

dynamically during inference with a large model (which requires substantial memory), the compressed model gives a fixed map due to their minimal size. Therefore, distillation can only help in reducing memory use at the model level; it is less flexible with regards to expert-level memory management that is needed by sparse architectures.

2.2.5 Neural Machine Translation by Jointly Learning to Align and Translate by Bahdanau et al. (2015)

To simplify neural machine translation, Bahdanau et al. (2015) introduced attention to the decoding process to enable a model to pay attention to relevant input tokens. Their recurrent networks calculate weights on alignment at each decoding step and this enhances the quality of translation and long sequences are better handled. Attention however, raises the usage of intermediate memory due to the usage of alignment matrices and stored hidden states. Although this inspired Transformer, it does not cause the size of the parameter memory to be smaller; all the model weights are kept loaded to inference.

2.2.6 Batch Normalization by Ioffe and Szegedy (2015)

Ioffe and Szegedy (2015) suggested Batch normalization which stabilizes and accelerates the training process through normalization of layer inputs across mini-batches. It causes a lower covariate shift inside the model and permits the greater learning rates resulting in quicker convergence and higher generalization. However, Batch Normalization is essentially training friendly; it does not reduce the number of parameters and memory imprint of the model. In perception, the acquired scaling and shifting parameters remain functional and the process does not foster the scalability of memory. Its primary contribution is performance with regard to trainings as opposed to the resource management on runtime mode.

2.2.7 Layer Normalization by Ba et al. (2016)

Two-dimensional normalization was introduced by Ba et al. (2016) as Layer Normalization that normalizes features activations, rather than batch ones. The method works better when batches statistics are volatile in recurrent and sequence models. Transformer architectures now incorporate Layer Normalization as a part of them. It increases training stability and convergence and does not lower the demands of inference run time memory; during execution all the model parameters are resident in the GPU memory. It, therefore, adds architectural stabilization, rather than memory optimization.

2.2.8 Gaussian Error Linear Units (GELU) by Hendrycks and Gimpel (2016)

Hendrycks and Gimpel (2016) proposed a combination of ReLU and stochastic regularization and called it the GELU activation function. GELU enhances the performance on the language and vision in Transformer feed-forward layers. Even while it has a positive impact on representational power, it does not alter the model size or memory footprint. Activation functions affect Churchill but not the need to load all weight matrices when making inferences. As such, its influence is on performance, rather than on the use of memory.

2.2.9 Attention Is All You Need by Vaswani et al. (2017)

The Transformer suggested by Vaswani et al. (2017) substitutes recurrent and convolutional sequence models with multi-headed self-attention and position-wise feed-forward networks. The architecture supports paralleling input token processing which enables high level of training efficiencies and scalability. Translate Experiments of machine translation demonstrate state-of-the-art by performance and lower training time than recurrent networks. Transformers however put all the parameters in memory at inference and therefore with increase in the model size, the memory requirements of the GPU also increase significantly. Although repetition bottlenecks are removed in the architecture, inference time memory scaling, which became important in large scale deployments was not addressed.

2.2.10 Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer by Shazeer et al. (2017)

Shazeer et al. (2017) proposed sparsely-gated Mixture-of-Experts layers, with the capacity to add parameter capacity but no expansion of per-token computation. Gating network chooses a small selection of experts per input whereas the experts allow conditional computation. It is shown that there is substantial improvement in language modeling and translation with computation manageable by experiments. However, not many experts are activated on a token, although the majority of systems maintain all expert parameters in memory, as part of inference. Thus, the use of memory depends on the number of total experts and not active ones. This separation of computational sparsity and memory density is one of the central points of later studies, such as CacheFlow.

2.2.11 Efficient Softmax Approximation for GPUs by Grave et al. (2017)

Grave et al. (2017) suggested effective softmax approximations in order to accelerate large-vocabulary language modeling on GPUs. They calculate the hierarchical clustering and adaptive partitioning method to minimize the computational complexity in the output layers. The training and inference time in experiments is faster than in standard softmax. Nevertheless, although the method is less costly in arithmetic, it does not reduce the requirement commonly required by all model parameters being resident at inference time. The focus of optimization is output-layer computation instead of the memory footprint and therefore is not concerned with the issue of weight residency in large Transformer or MoE models.

2.2.12 Improving Language Understanding by Generative Pre-Training by Radford et al. (2018)

The article by Radford et al. (2018) presented a new method of NLP in generative pre-training followed by fine-tuning as a potent tool. They use large language models based on transformers, which they trained on large unlabeled corpora and distilled to downstream with minimal learning. The tests revealed significant gains in such benchmarking as question answering and textual entailment. The designs are based on dense Transformer layers, where all the parameters are opened during inference. The model supported the tendency of increasing the performance with the number of parameters, although it was smaller than subsequent large systems. It greatly developed transfer learning in NLP but failed to take into account memory limitations when running at runtime and how to manage the parameters dynamically.

2.2.13 Breaking the Softmax Bottleneck by Yang et al. (2018)

Yang et al. (2018) discovered the so-called softmax bottleneck, the constraint of expressiveness of typical softmax layers in language models. They put forward a high rank factorization that upscales representational capacity but at a minimal computational cost. On language-modeling benchmarks, it was found that there was a decreased perplexity. The approach, though, does not decrease the number of stored parameters or modify assumptions about memory residency: all of the learned matrices have to be available in order to perform inference. In turn, inference-time memory pressure, particularly that of large Transformer models, does not decrease, as modeling capacity is improved.

2.2.14 Language Models are Unsupervised Multitask Learners by Radford et al. (2019)

.

GPT-2 was presented by Radford et al. (2019), who showed that the zero-shot setting allows large-scale language models which have been trained on a variety of web data to handle a number of tasks. The model takes a deep Transformer decoder that has much more parameters than previous systems. The findings indicate high performance in translation, summary and question answering without fine-tuning. Yet it is a completely dense architecture and must have full parameter residency on inference. The size of the model (billions of parameters) renders the use of GPUs memory intensive, which tends to increase deployment costs due to memory limits.

2.2.15 Generating Long Sequences with Sparse Transformers by Child et al. (2019)

To reduce the quadratic cost of self-attention, Child et al. (2019) presented Sparse Transformers. They encode orderly sparsity in attention matrix that allows the sequential processing of long sequences at a lower rate of memory and computation. Better results in experiments on long context language models are demonstrated. This low density reimbursement attention-related memory artifact, but not residency of model weights. In inference, all the feedforward and projection parameters are loaded in their entirety. Therefore, attention sparsity helps in alleviating sequence-length scaling, but does not tackle the bigger problem of expert weight memory in large MoE systems.

2.2.16 Root Mean Square Layer Normalization by Zhang and Sennrich (2019)

Zhang and Sennrich (2019) suggested a simplified version of Layer Normalization called Root Mean Square Layer Normalization (RMSNorm). As computational efficiency and numerical stability RMSNorm is a variant of mean-centered variance-normalized normalization. Tests have demonstrated performance similar and better than standard Layer Normalization in Transformers. RMSNorm is a computationally less costly but does not modify the overall number of the parameters and the memory residency of the parameters in the inference. It makes its contribution in optimization and stability rather than scalability of the memory usage. Even large Transformer and MoE models based on RMSNorm require full weight residency.

2.2.17 Language Models are Few-Shot Learners by Brown et al. (2020)

GPT architecture Transformers were expanded to 175 billion parameters, presented by Brown et al. (2020). The primary contribution is that the demonstration of powerful few-shot and zero-shot learning without either fine-tuning or scaled down to huge sizes. The dense decoder is trained using huge datasets using distributed infrastructure. Findings depict significant improvements on NLP benchmarks (question answering and translation). The dense structure however requires full parameter residency to be applied in inference and therefore cannot be deployed locally with specialized clusters. Although GPT -3 confirms scaling laws with respect to performance, it illustrates drastic limitations of dense large-scale models in memory.

2.2.18 Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer by Raffel et al. (2020)

Raffel et al. (2020) suggested T5, which resolves all NLP tasks to a single text-to-text model. The structure is built based on a standard Transformer encoder-decoder that is trained on a huge corpus that is under curation. It shows high transfer-learning on various benchmarks. Although T5 enhances training methodology and task unification, the subnet has a dense execution of parameters that necessitate an entire model reservoir throughout the inference phase. When T5 is scaled up, the amount of memory required by the GPU is scaled up accordingly. The work makes progress on modeling generality but no progress on inference time memory constraints.

2.2.19 DeepSpeed: System Optimizations Enable Training Deep Learning Models with Over 100 Billion Parameters by Rasley et al. (2020)

Rasley et al. (2020), proposed system-level DeepSpeed, which is an efficient training of extremely large models by optimization of memory and distributed partitioning. The ZeRO (Zero Redundancy Optimizer) of DeepSpeed divides optimizer states and gradients in the devices which allows less memory duplication through training. Experiments demonstrate that it is possible to train models of more than 100 billion parameters on commodity GPU clusters. DeepSpeed is however mainly aimed at training efficiency and distributed scaling and not optimization of single node inference. Dense models even during inference still need full parameter residency, unless they are specifically partitioned. DeepSpeed cuts, therefore, reduce but not eliminate, the redundancy of memory accessible to of-the-shelf inference on a single gpu.

2.2.20 An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale by Dosovitskiy et al. (2020)

Dosovitskiy et al., (2020) proposed the Vision Transformer (ViT) that uses the Transformer architecture to deal with image recognition by splitting an image into patches that can be regarded as tokens. The model is able to equal convolutional neural networks on massive datasets, interface attention-based designs outside of NLP. Similarly to language Transformers, ViTs use dense parameter network models which also require full memory residency during inference. The larger model depth and width the more memory memory of a graphics card also require. Although the work extends the use of Transformer, it does not change the memory scaling properties.

2.2.21 GLU Variants Improve Transformer by Shazeer (2020)

Gated Linear Unit (GLU) variants were presented by Shazeer (2020) as variants of Transformer feed-forward layers focused on increasing their expressiveness. These variants introduce gating mechanisms to linear transformations and hence the variants are able to better skilled modeling capacity and enhance training stability. Experiments indicate that GLU versions do better with a variety of language-modeling tasks than do the normal feed-forward layers. The changes also, on some parameters, increase computational load and parameter count. Notably, there is no net reduction in inference-time memory GLU does not require the use of memory to decrease inference-time Memories: all transformed feed-forward parameters have to remain in memory during model execution, which is a dense-memory assumption.

2.2.22 Longformer: The Long-Document Transformer by Beltagy et al. (2020)

Proposed by Beltagy et al. (2020), the Longformer is the Transformer dispensing the entire idea of self-attention with a combination of local windowed attention and sparse global attention. The method reduces the quadratic memory and calculate cost of attention on long documents. Longformer has also been experimentally found to scale to long-sequence tasks, such as document classification and question answering, much better. Longformer is more economical in terms of attention-buffer memory, but the fact remains that not only do all model weights remain loaded when making inferences, but they are also held there. Attributes that represent the attention activations still take a backseat compared to the parameters in terms of occupying the GPU memory footprint.

2.2.23 ALBERT: A Lite BERT for Self-Supervised Learning of Language Representations by Lan et al. (2020)

Lan et al. (2020) proposed ALBERT, a downsized version of BERT, which downsizes model size without degrading performance through sharing the weights across the layers and factorizing the embedding matrices. These transformations decrease significantly the number of parameters total as compared to BERT. It has been experimentally demonstrated that ALBERT obtains competitive accuracy with less memory consumption. Nevertheless, it is the architectural compression, rather than the dynamic run time control, that is remembered. ALBERT reduces the number of parameters which are always static but does not have adaptive memory scheduling or conditional expert activation. As such it is a compression method as opposed to a runtime inference optimizer.

2.2.24 GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding by Lepikhin et al. (2021)

Lepikhin et al. (2021) introduced GShard, which is a system that scales massive models with conditional computation as well as automatic sharding of parameters introduced in distributed hardware. It uses sparse layers of Mixture-of-Experts on Transformers and separates experts among modified devices so as to balance memory and computing duties. Experiments indicate that GShard has the ability to train very large MoE models on distributed clusters of TPUs. But the method is directed towards multi-device situations that already possess distributed memory. It makes no effort inference-time memory constraints on one GPU. Special weights remain full in the partition distributed and soft runtime scheduling at hard memory constraints is not investigated.

2.2.25 Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity by Fedus et al. (2022)

Fedus et al. (2022) proposed the Switch Transformer, that is, Mixture-of-experts routing is simplified through only one expert per token (top-1 routing) rather than using many experts. This minimizes the routing overhead and enhances stability of the training, enabling models to scale to trillion parameters. Experimental optimization has shown that Switch Transformers are equal to dense models in terms of cost per token. Nevertheless, the inference systems usually maintain all expert parameters

in memory to support dynamic routing and therefore the memory consumption is directly proportional to the number of experts as opposed to the quantity of experts that are actually active. In this way, even though Switch Transformers can promote sparse scaling, they have no implication of automatically addressing inference-time memory residency issues.

2.2.26 FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness by Dao et al. (2022)

FlashAttention is an algorithm demonstrated by Dao et al. (2022) as it reduces memory overhead and accelerates self-attention. Focusing on memory-access patterns and being IO-aware, FlashAttention makes very precise attention calculations and at a far smaller memory footprint. Standard experiments demonstrate an increased throughput and low-memory consumption associated with processing long sequences in contrast to the usual attention. However, FlashAttention is concerned with the activation memory rather than the model parameters that are static. If it does not lower the need to have all the model weights such as expert weights in memory. Thus, FlashAttention can ease scale-to-sequence but is not a more powerful way of overcoming the expert-weight bottleneck in memory.

2.2.27 FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning by Dao (2023)

FlashAttention was further improved to a more-parallel organization and workload partitioning by Dao (2023) to create FlashAttention 2. It actively serving device is increasing the usage of the graphics card, reducing synchronization costs, which translates to additional performance and memory-saving improvements over the base. Similarly to FlashAttention, FlashAttention-2 does not focus on the parameter storage of Transformers or MoE models but instead, intermediate activation memory and compute efficiency. It therefore enhances the performance of the process of attention execution but fails to address the memory residency at the expertise level among sparse architectures.

2.2.28 Efficient Memory Management for Large Language Model Serving with PagedAttention by Kwon et al. (2023)

Kwon et al. (2023) proposed the memory-based attention method, which is PagedAttention, which can improve the efficiency of large-language-model serving. The

technique virtualizes key-value (KV) cache storage using paging of memory instead of consecutive blocks that make it be less prone to fragmentation and better utilize the GPU. Experiments demonstrate that memory waste and throughput are less in the case of long-context inference. Yet, PagedAttention only deals with the memory in terms of states of attention, as opposed to the model parameters. It is not a solution to maintaining all model weights and, in particular, expert weights in MoE models, in GPU memory. In this way, the technique is a more efficient way of serving, but does not eliminate expert-level memory bottlenecks.

2.2.29 LLaMA: Open and Efficient Foundation Language Models by Touvron et al. (2023)

Touvron et al. (2023) presented a family of open and relatively efficient dense Transformer-based models, the use of which uses optimized strategies of data curation to propose LLaMA. The researchers demonstrated that intelligently developed training pipelines could be used to perform equally well with respect to larger systems using smaller models. Results of the experiment were good on various NLP benchmarks. Nevertheless, LLaMA is a large model that loads all the parameters in memory when performing inference. It also scales its memory requirement with the size of models, limiting its use to consumer GPUs. Thereby, although LLaMA is more efficient in terms of parameter optimization compared to the previous dense models, it lacks the capability to handle dynamic memory management.

2.2.30 SwapMoE: Serving Off-the-Shelf MoE-based Large Language Models with Tunable Memory Budget by Yoo and Lee (2023)

SwapMoE, suggested by Yoo and Lee (2023), is a system tapping expert weights in both CPU and GPU to allow the inference of large MoE models and limited memory. It has a memory limit fixed by the user and automatically transfers experts at on-demand. It was found that experiments indicated lower peak GPU memory and inference with stricter VRAM limits. Swapping is based upon heuristic/LRU-based policy and neither exploits fine-grained context nor any key-value reuse data. The round-trip transfer of CPU to and from GPUs may introduce a latency and stability may be compromised under intermittent loads. SwapMoE deals somewhat with expert memory residency and does not provide very advanced scheduling to reduce thrashing.

2.2.31 MoE-Infinity: Efficient MoE Inference on Personal Machines with Sparsity-Aware Expert Cache by Zuo et al. (2023)

The article by Zuo et al. (2023) proposes MoE-Infinity that leverages expert caching based on the idea of sparsity to execute MoE inferences on personal computers. The system evicts and loads experts depending on patterns of access, reducing the requirements of the GPU memories. Tests demonstrated that MoE models in large scale could be executed on commodity equipment which had managed memory consumption. The defaulting cache is based on basic eviction brush strokes and does not utilize any token level context or key-value reuse indications. There could be communication overhead in the event of frequent repeated transfers between memory tiers. MoE-Infinity builds upon memory-aware MoE inference, but does not give the scheduling logic the continuity as a forward path of the model.

2.2.32 DeepSpeed-MII: Model Inference Interface by Microsoft (2023)

DeepSpeed-MII offers an inference interface that runs on DeepSpeed distributed optimization infrastructure, that allows serving large-model applications simultaneously using multiple GPUs. The system is based on model parallelism and pipeline using optimized inference to enhance throughput. It is not aimed at fine-grained expert-limited node level scheduling of MoE architectures, but at deploying clusters on a large scale fulfilling single-node memory constraints. Dense parameter residency is the default. DeepSpeed-MII advances the inference scale in a multi-device design, but is not concerned with the expertise weight memory management of restricted GPUs.

2.2.33 TensorRT-LLM: Optimized Inference for Large Language Models by NVIDIA (2023)

TensorRT-LLM is a highly optimized inference engine bringing the use of kernel fusion, optimization of precision and hardware acceleration to improve the performance of large language models. It minimizes the latency and maximizes throughput by the way of low-level optimization and efficient utilization of GPUs. Choose any model However, TensorRT-LLM supposes that all model weights are resident in AV RAM prior to execution. It does not have active expert scheduling or weight-switching of MoE architectures. Although it accelerates the computation process, memory footprint is still associated with size of total parameters. In this way, TensorRT-LLM

is maximizing speed, without addressing the main memory scale issue of large systems using MoE.

2.2.34 DeepSeek-V2: A Strong, Economical and Efficient Mixture-of-Experts Language Model by Liang et al. (2024)

Liang et al. (2024) came up with DeepSeek-V2, a massive Mixture-of-Experts language model that is more efficient in the use of parameters but equally does not compromise its performance. The architecture amalgamates distant expert layers to the Transformer backbone and concentrates on economical scaling in order to reduce the computational cost. Competitive accuracy was demonstrated to be better than dense models of approximately similar effective capacity and greater compute efficiency per token. But inference typically stores all expert weights in which to run dynamic routing are stored in the GPU memory. Therefore, DeepSeek-V2, however, makes scaling efficient, but fails to address the problem of memory residency on limited hardware.

2.2.35 fMoE: Fine-Grained and Prompt-Aware Expert Offloading for Large Mixture-of-Experts Serving by Shen et al. (2024)

Shen et al. (2024) developed fine-grained, prompt-sensitive expert offloading fMoE, an expert offloading mechanism to help to serve the needs of large Mixture-of-Experts model. The system is used to predict expert execution and purges inactive experts to identify prompt features, minimizing the peak memory in the GPU. It was experimentally demonstrated that memory efficiency was enhanced and fewer swaps were occurring compared to the case with the use of a static offloading. fMoE targets distributed serving environments and balances performance across devices. Even though it implements prompt-awareness, it does not schedule on a single GPU such that it is a strict part of the forward path of the model. Coordination overhead and routing analysis can be imposed by high concurrency on latency; fMoE is an expert aware offloading technology that does not consider context offloading with strong single node memory constraints.

2.3 Comparative Summary of Reviewed Systems

Mixture-of-Experts, dense Transformers, scaling innovations and inference-serving systems are included in the literature. This is despite the fact that these studies enhanced modeling, efficiency and scalability but their approaches to its management

during the inference process vary widely. Very limited amount of systems are concerned with the maintenance of expert weights on the GPU in the event of limited memory. The following table gives the sparsity, expert granularity, memory strategy, deployment focus and inference-time memory constraints of each system.

Analytical Interpretation

The most important patterns in the analysis:

- Dense Architectures and Static Residency:

  Most Transformer-based models store all the parameters in the GPU memory during inference. The weights remain resident, although they can be made cheaper to train, or they can be made much smaller during activations, preventing scalability.

- Sparse Compute vs. Sparse Memory:

  The models of MoE minimize the calculation by utilizing a small number of experts on each token. However, in most instances, all the experts remain loaded even when not in use hence, memory sparsity is nonexistent.

- Offloading-Based Solutions:

  SwapMoE and MoE-Infinity reduce peak VRAM memory by swapping of both the GPU and CPU memory. Such schemes are based on heuristics or LRU eviction and may bring in transfer delays.

- Distributed-Centric Designs:

  GShard and DeepSpeed are concerned with the optimization of multi-device partitions, as opposed to single-GPU. They also assist in groups but not in a single constrained node.

- Limited Context-Aware Scheduling:

  Very little systems take advantage of token-level context or key-value reuse to make decisions on which experts to retain. The Majority make use of static rules or generic caches.

On the whole, the community came a long way in terms of the computational power and scaling, yet there is still nothing about finer/grained/context-aware work-scheduling on a single-GPU deployment.

Table 2.1: Comparative Summary of Reviewed Architectures and Systems

Related Work	Sparse Compute	MoE Layer	Expert Scheduling	CPU-GPU Swap	Consumer GPU Ready	Memory Limitation
Adaptive MoE (1991)	Yes	Yes	No	No	N/A	No runtime memory management considered.
Transformer (2017)	No	No	No	No	Yes	Full-weight residency required at inference.
Sparsely-Gated MoE (2017)	Yes	Yes	No	No	No	Experts typically remain fully resident.
Switch Transformer (2022)	Yes	Yes	No	No	No	Sparse compute; memory dominated by expert weights.
FlashAttention (2022)	No	No	No	No	Yes	Optimizes attention activations, not weight residency.
PagedAttention (2023)	No	No	No	No	Yes	KV-cache paging only; expert weights unchanged.
SwapMoE (2023)	Yes	Yes	Partial	Yes	Partial	Transfer overhead and thrashing under frequent swaps.
MoE-Infinity (2023)	Yes	Yes	Partial	Yes	Partial	Heuristic caching; limited context prioritization.
DeepSpeed-MII (2023)	No	No	No	Partial	No	Distributed focus; not single-GPU expert scheduling.
TensorRT-LLM (2023)	No	No	No	No	No	Assumes weights already resident in GPU memory.
DeepSeek-V2 (2024)	Yes	Yes	No	No	No	Does not inherently solve constrained expert residency.
fMoE (2024)	Yes	Yes	Partial	Yes	Partial	Designed for multi-device serving; coordination overhead.

2.4 Research Gap

Architecture (above), efficiency and scaling The above literature review demonstrates advancements in architecture, performance and size. Conditional computation was introduced by Mixture-of-Experts in order to increase model capacity without proportionately increasing compute. Parallel sequence modeling would then be made possible by transformers and very large models such as GPT-3 had proved these scaling laws. Recent research addresses sparse routing, sharding, memory-efficient attention and expert offloading to reduce compute and memory bottlenecks.

Nevertheless, a number of gaps are left open to inference of the limited GPU memory:

- Dense models necessitate the weights taking up all the weight, which is not feasible on consumer-grade devices.
- MoE continues to maintain all the experts loaded and thus memory is a function of the number of total experts and not the active experts.
- The offloading systems are coarse-grained, heuristic driven systems; they might provoke the frequent transfers of data, leading to latency and thrashing and are oriented to the distributed systems.
- Attention optimizations do not just decrease the memory of the activations, but they primarily do not decrease the storage of the expert weights.

The literature taken together indicate a discrepancy: memory sparsity is not accompanied with arithmetic sparsity. Only a small number of systems take into account fine grained and context sensitive scheduling during the forward pass on one GPU with hard constraints. Expert weights are so far treated by few frameworks and ranked in order by reuse patterns that they can maintain their execution without architecture modification. Such a gap is the inspiration of memory aware scheduling of MoE inference in CacheFlow.

2.5 Summary

Chapter 2 provided a summary of scalable neural models, starting with ancient Mixture-of-Experts to contemporary inference serving models. Mixture-of-Experts This paper presented conditional computation; Transformers presented parallel sequence modeling, scaling tools such as sparse routing and sharding had increased

compute efficiency, but could do nothing about creating inference-time memory constraints. Subsequent literature covered compression, parameter sharing, efficient softmax and activation optimizations. Newer systems introduced memory-aware algorithms, such as KV-cache paging, CPU-GPU swapping, partitioning, although most of them still assume complete expert residency.

Computational sparsity in sparse MoE is not associated with memory sparsity, limiting the memory of GPUs. Those systems are primarily based on dense model acceleration, scaling distribution, or heuristically conducted offloading. Very limited support fine-grained, context-driven scheduling into a single-gpu execution path. This continuous gap leads to the requirement of CacheFlow, which is a framework that dynamically supports expert presence, according to run-time requirement and ensures proper execution.

The second chapter is based on this premise by stating the proposed system architecture and temporal scheduling approach of CacheFlow that would resolve the mentioned limitations of inference-time memory management of Mixture-of-Experts models.

47

Chapter 3

Methodology

3.1 Introduction to the Proposed Architecture

The CacheFlow system is built to facilitate Mixture-of-Experts (MoE) inference with extreme memory constraints on GPU hardware. The fundamental implementation is deeper than mere GPU-based cache: a novel three-level memory hierarchy where disk-based expert weights are smoothly transferred to staging the pinned CPU RAM at the GPU VRAM slots via non-blocking, asynchronous PCIe transfers. This architecture essentially separates graphics memory demands and the overall count of parameters in an expert model, allowing execution of huge MoE models on hardware with limited resources.

The suggested method is an optimization at the system level, which does not alter the original MoE architecture or the structure of the transformer models. Rather, it runs on the layer level by replacing conventional parallel expert modules with an optimized implementation which has built-in direct disk I/O, smart memory staging, asynchronous pre-fetching and temporal cache freezing functionality. Instead of storing all expert weights on host RAM (as in traditional solutions), CacheFlow takes advantage of the fact that expert weight files are compactly stored on the disk in the safetensors format. This system directly reads the necessary expert weights on-demand out of disk, stages them via CPU-pinned memory buffers and asynchronously transfers them to GPU slots. An advanced caching scheme that intra-batch freezes keep common experts in GPU memory and cold experts loadable and able to be accommodated with a fixed capacity constraint.

Such a three-tier architecture (Disk, Pinned RAM and GPU VRAM) forms a memory-performance trade-off space in which capacity can be configured on a per-application basis. The system is computationally correct and experiences dramatic memory footprint cuts on the GPU and allows previously unreachable models to be run by resource-constrained systems.

3.2 System Architecture and Memory Pipeline

3.2.1 Three-Tier Memory Hierarchy

The CacheFlow system clearly defines the storage and access of expert weights in three tiers of memory:

Tier 1: Disk (Persistent)

- All the expert weight matrices are stored authoritatively
- Expert weights are stored in the safetensor format which is a compressed binary serialization format that allows random access of elements
- The access is implemented through MmapExpertStore, which does a zero-copy slicing of expert weight tensors
- Large capacity (model-dependent, usually 10-100 GB in the case of large MoEs)
- Low access bandwidth (100-500MB/s -based on storage medium)
- None, no GPUs involved: weights are read off disk by CPU

Tier 2: Pinned Host Memory (Pinned Staging Buffer)

- Fixed size collection of CPU-resident memory buffers with page-locking semantics of CUDA
- Used as a temporary loading point of weights in disk
- The weight matrices of each staging buffer are that of one expert
- Pinned allocation enables asynchronous and zero-copy PCIe transfers to GPU without software involvement
- Typically allocated with torch.cuda.pin_memory()
- A hyperparameter that can be adjusted is number of staging buffers (staging_slots) (e.g., 8 or 16)
- Provides a decoupling point between the slow disk I/O and the fast GPU transfer

Tier 3: GPU VRAM (Working Set)

- Active computation and inference is reserved
- Has: model static parameters (embeddings, attention layers, output head), intermediate activations and a fixed set of expert weight slots in GPU memory

- Smaller capacity of between 6 GB to 24GB on consumer grade hardware
- Very high-bandwidth access (up to 900 GB/s on current NVIDIA GPUs)
- Expert slots are executed as a pre-allocated torch.Shape:Tensor of shape (num_gpu_slots, output_size, input_size)

3.2.2 Direct Disk I/O through MmapExpertStore

The MmapExpertStore class is an abstraction of expert weight access to disk storage. In contrast to conventional methods which store complete weight matrices in RAM, MmapExpertStore uses a slice-based method, which is memory-efficient:

Initialization Phase:

store = MmapExpertStore(weight_file)

The caller of read_metadata checks the safetensors file header to identify:

- The tensor key of expert weights (usually, model.experts.weight, or similar)
- The shape of the tensor (num_experts, output_size, input_size)
- The type of data in the tensors (float32, float16 or int8)

The complexity of this metadata reading is $O(1)$ and it only needs to check the headers; no weight data is read into memory.

Per-Expert Loading:

expertweight = store.loadexpert(expertid, out=stagingbuffer)

Load expert method follows the steps as follows:

1. **Bounds Checking:** Check that $0 \leq$ expertid < numexperts
2. **Safetensors Slicing:** Open the safetensors file and call get_slice() with only the weight matrix number of expertid. The slice operation of safetensors retrieves an exact outputsize by inputsize by dtype_bytes bytes of data on disk, ignoring the data of previous experts
3. **Buffer Population:** Transfer the sliced weights to the given out buffer (a pinned memory allocation). This copy operation involves effective CPU memory operations but does not send data to GPU at this point
4. **Return:** Publish the filled-in buffer tensor

Its major strength lies in the fact that disk I/O is proportional to the size of a single expert and not the weight file as a whole. On a model with 60 100 MB experts (6 GB in total) loading a single expert requires only 100 MB of disk.

3.2.3 Staged Pinned Memory and Asynchronous PCIe Transfer

In GraniteMoeParallelExperts the nextstaging() method is used to control the lifecycle of pinned memory buffers:

```
def nextstaging(self) -> Optional[torch.Tensor]:
    # Gets a free pinned buffer, or blocks until one is free.
    # Once the disk I/O of the buffer is made, it is immediately scheduled to
    # asynchronous transfer to graphics card. The system is awaiting previous
    # transfers when necessary.
```

Staging pipeline works in the following manner:

1. **Buffer Availability Check:** Have a list of pinned buffers. When buffers are in-flight (waiting PCIe transfers), the system blocks till it finishes
2. **Synchronization Point:** It is important to be sure that any previous PCIe transfer that used the same buffer has completed before reusing it. This is monitored through torch.cuda.Event objects
3. **Return Buffer:** Provide a ready pinned buffer to be used in the next disk read operation

Once a disk weight has been loaded into a pinned buffer, a scheduled asynchronous PCIe transfer is scheduled using torch.cuda.Stream:

```
stream = torch.cuda.Stream()
with torch.cuda.stream(stream):
    gpuslot[:] = pinnedbuffer       # DMA through PCIe
    event = torch.cuda.Event()
    event.record(stream)    # Record completion in stream
    pending[expertid] = (event, gpuslot)    # Follow through pending transfer
```

This design enables loading of the weights of multiple experts on disk and transmitting them to the GPU simultaneously. As the CPU is reading the weights out of disk, the weights of Expert B that were previously loaded into the graphics card via the PCIe process are being sent to the graphics card and the computation of Expert C is being performed on the graphics card. Overall latency is dramatically reduced by this overlapping of I/O, transfer and computation.

3.3 ExpertCache: Freezed Cache on GPU Residents

The ExpertCache class handles a fixed size LRU cache of experts in GPU memory. In addition to typical LRU operations, it also includes a critical freezing environment to eliminate intra-batch conflicts:

Cache State:

- cache: OrderedDict[int, torch.Tensor]: Converts expert IDs into their associated weight tensors on the GPU
- _frozen: set[int]: Set of expert IDs which may not be evicted (locked out in current batch processing)
- capacity: int: The maximum number of experts that can be kept in the cache at a time

Freezing Mechanism:

```
def freeze_many(self, expertids: List[int]) -> None:
    self._frozen.update(expert_ids)

def unfreeze_many(self, expert_id: List[int]) -> None:
    for expert_id in expert_ids:
        self._frozen.discard(expert_id)
```

When forward passing, the list of experts required (by token routing) is frozen in the cache at the start of the pass. This guarantees that when the system needs to fetch an expert which is not in the cache (a cold expert), it does not mistakenly evict a needed expert which is in the cache. Freezing is set free at the completion of the batch and normal LRU eviction is carried out on the succeeding batch.

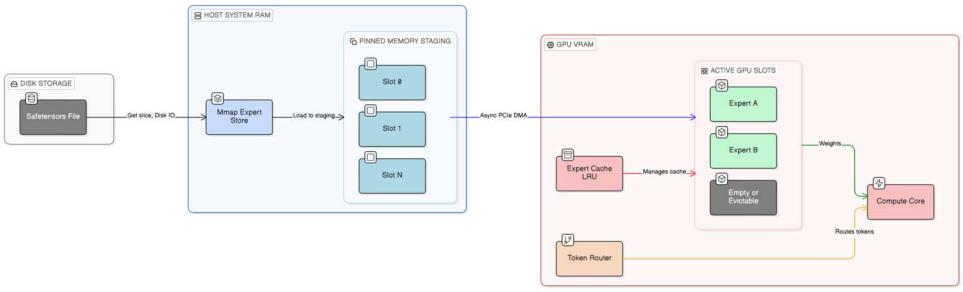

Figure 3.1: The three-tier CacheFlow memory hierarchy illustrates the complete data pathway from persistent disk storage through pinned CPU memory staging to GPU VRAM. Expert weights are stored compactly in safetensors format on disk. The MmapExpertStore reads specific expert slices on-demand. Pinned memory buffers stage the weights for asynchronous transfer. The GPU cache maintains the working set of experts. The diagram shows multiple experts at different stages: some resident in GPU cache, some pending transfer to GPU and some available for disk loading.

3.4 CacheFlow Routing Mathematically Modeled

3.4.1 Memory Budget Decomposition using Three-Tier Architecture

The per-user GPU memory usage of an optimized model of MoE is partitioned into the three-level hierarchy:

$$M_{VRAM} = M_{static} + M_{cache} + M_{KV} + M_{act}$$

Where:

- M_{static}: Memory used by the model parameters (embeddings, attention layers, output head, layer normalization) that do not change and are stored in the GPU memory permanently
- M_{cache}: Memory used by expert weights in the VRAM of the graphics card (the working set)
- M_{KV} : Cache memory of key-values in attention (where needed)
- M_{act}: Memory of intermediate activations in forward computation

The key innovation here is that the cache memory does not depend on the overall number of experts. The cache memory is controlled by:

$$M_{cache} = L_{MoE} \times C \times P_{expert} \times b_o$$

Where:

- L_{MoE}: MoE layers in the model
- C: Capacity constraint - the largest number of expert slots in GPU VRAM per layer (i.e. $C = 4$ implies that at any time at most 4 experts can be resident in the GPU VRAM on each layer)
- P_{expert}: The size of single expert weight matrix (in terms of number of parameters)
- b_o: The size of a parameter (usually 2 bytes (float16) or 4 bytes (float32))

3.4.2 Host Pinned Staging Memory Footprint

The staged buffer of pinned memory is unique to both the GPU memory and disk storage:

$$M_{Pinned} = L_{MoE} \times S_{staging} \times P_{expert} \times b_o$$

Where:

- $S_{staging}$: The quantity of staging buffers (e.g., $S_{staging} = 8$)

More importantly, $S_{staging}$ is often significantly less than the number of experts (S_{slots}). While S_{slots} may be 60 or more, $S_{staging}$ is often 4–16. With this design the system is capable of pipelining disk I/O and PCIe transfers without proportional allocation of pinned memory.

3.4.3 Disk Storage and the Entire Memory Budget

The disk storage is limited and proportional to the number of the overall experts:

$$M_{Disk} = L_{MoE} \times S_{slots} \times P_{expert} \times b_o$$

Where:

- S_{slots}: The number of experts in the entire MoE layer (e.g., 60 in the Qwen2-MoE model)

This is done to complete the three-tier budget:

$$M_{Total} = M_{VRAM} + M_{Pinned} + M_{Disk}$$

The critical decoupling is observed:

- VRAM is only dependent on C (GPU capacity constraint)
- The value of M_{Pinned} only depends on the number of staging buffers ($S_{staging}$)
- The disk size is based on the number of slots on the disk (S_{slots}) and is not limited by hardware

3.4.4 Memory Reduction Formula

CacheFlow supports the implementation of large expert models on GPUs with memory constraints: by setting capacity constraints C and staging buffers $S_{staging}$, it is possible to deploy large expert models on constrained GPUs:

$$\text{VRAM Reduction} = 1 - \frac{C}{S_{slots}}$$

For example, with $C = 4$ and $S_{slots} = 60$:

$$\text{VRAM Reduction} = 1 - \frac{4}{60} = 93.3\%$$

This is a 93.3% cut in the amount of expert-related GPU memory usage over loading all experts at once.

3.4.5 Dynamic Cache Buffer Implementation

The CacheFlow dynamic buffer assigner can be mathematically formulated as:

$$M_{cache} = L_{MoE} \times C \times P_{expert} \times b_o$$

The footprint of host RAM pinning is:

$$M_{Pinned} = L_{MoE} \times S_{staging} \times P_{expert} \times b_o$$

These equations alone demonstrate how the system can be decoupled between the overall number of parameters and the VRAM requirement through the control of the capacity C.

3.4.6 Token Routing and Hit Efficiency of Caches

The MoE routing algorithm is used to select the experts to process a token. Gating network makes a distribution of all experts given a single input token. The routing usually picks the K experts using each token (e.g. $K = 2$) according to gating logits.

The team of experts that will be needed in a batch is set as:

$$\text{Required\_Experts} = \{i : \text{expert\_size}[i] > 0\}$$

The productivity of CacheFlow is directly proportional to the overlap between the needed professionals and the capacity of the cache:

$$\text{Cache Hit Rate} = \frac{|\text{required\_experts} \cap \text{cached\_experts}|}{|\text{required\_experts}|}$$

Expert access patterns exhibit temporal locality, where recent tokens prefer to leave a path to similar sets of experts and this is used to achieve the highest hit rates with a given capacity C.

3.5 Asynchronous Execution Flow

3.5.1 Forward Pass Initialization and Batch Setup

When a forward pass is called on a GraniteMoeParallelExperts module the following initialization sequence is executed:

1. **Routing Distribution Computation:** The gating network uses the input tokens to compute expert routing logits and generates a distribution over experts
2. **Identification of Experts Needed:** The list of experts needed by the current batch is identified:

   ```
   required_experts = [i for i, size in enumerate(expert_size) if size > 0]
   ```

   The input is divided into parts (input tensors) in this operation through expert assignment to produce sub tensors per expert
3. **Cache Freezing:** The cache is pinned down with all the necessary professionals lest they be evicted by accident:

   ```
   cache.freeze_many(required_experts)
   ```

   This is so that as the cold experts are fetched, the system will not evict any needed experts that are already wrapped up
4. **Residency Classification:** The system partitions required experts into two classes:
   - **Cached Experts (Cache Hits):** `cached_experts = [e for e in required_experts if e in cache.cache]`
   - **Cold Experts (Cache Misses):** `cold_experts = [e for e in required_experts if e not in cache.cache]`

3.5.2 pre-fetching and Pending Transfers Asynchronous

To cold specialists, the system starts asynchronous loading:

```
pending = {}    # expert_id -> (torch.cuda.Event, gpu_slot)
```

The following pipeline is launched to each cold expert:

Disk Read (CPU): Read in a staging buffer that is available and load the weights of the expert on disk:

```
staging_buffer = next_staging()      # Block when all buffers have been started
store.load_expert(expert_id,  out=staging_buffer)
```

The call load_expert() reads the expert slice on disk with safetensors and blocks the CPU thread until the read is finished.

Asynchronous Transfer Scheduling (GPU): Schedule non-blocking PCIe transfer of pinned memory to GPU:

```
stream = torch.cuda.Stream()
with torch.cuda.stream(stream):
    gpu_slot[:] = staging_buffer      # Non-blocking transfer to GPU slot
    event = torch.cuda.Event()
    event.record(stream)      # Record completion event
    pending[expert_id]  =  (event,  gpu_slot)
```

The point to note is that gpu_slot[:] = staging_buffer is asynchronous; the DMA engine of the GPU is involved in the transfer without blocking either the CPU or the GPU compute. The system does not wait till it is completed.

Overlap with Computation: As the weights of cold experts are being transferred:

- At the GPU, the computations of the forward pass of the cache experts commences
- Other weight weights of cold experts may have their disk reads started by the CPU
- There are several transfers of multiple experts simultaneously through PCIe

3.5.3 Before Expert Computation Synchronization

The system should make sure that the weights of a cold expert have been loaded into GPU memory even before calculating its contribution:

```
if expert_id in pending:
    event, gpu_slot = pending[expert_id]
    event.synchronize()  # Wait until PCIe transfer has completed
    out_chunks[expert_id] = F.linear(input_list[expert_id], gpu_slot)
    del pending[expert_id]
else:
    # Expert already was etched; calculate straight off
    expert_weight = cache.cache[expert_id]
    out_chunks[expert_id] = F.linear(input_list[expert_id], expert_weight)
```

The event.synchronize() call prevents the CPU from running until the DMA transfer of the GPU is complete. This makes sure that weights of the expert are in GPU memory before computation can commence.

3.5.4 Execution Ordering to Reduce Latency

The execution order will be selected in such a way that wait time will be minimized:

```
execution_order = cached_experts + cold_experts
```

This order guarantees that:

- Expert cache requests are done instantly (no delayed transfer)
- Cold experts would be processed in sequence once their transfers have been completed asynchronously
- When the system gets to a cold expert, all possible time has passed to get the weights off disk to staging to GPU

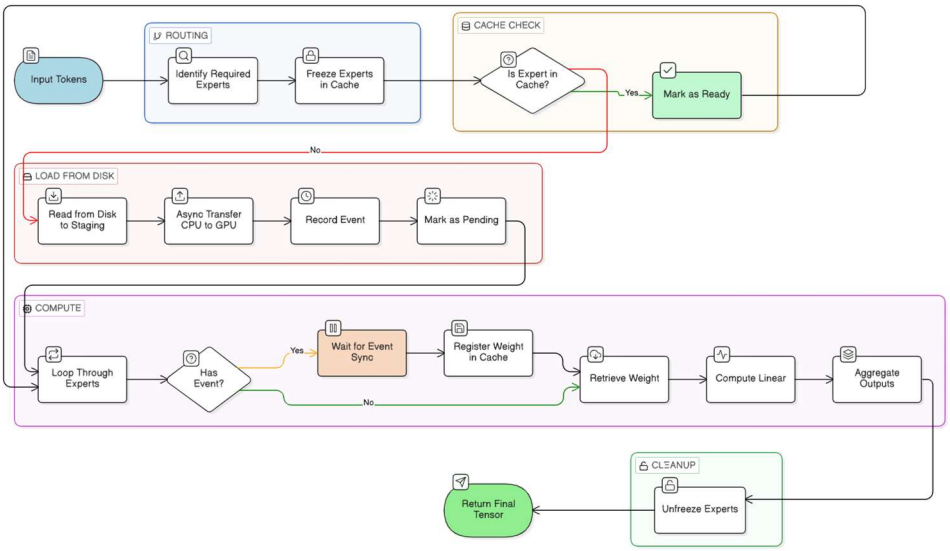

Figure 3.2: The asynchronous execution flow demonstrates the overlap of disk I/O, PCIe transfer and GPU computation. At Time T0, Expert A is being read from disk, Expert B is being transferred via PCIe and Expert C is being computed on the GPU. The pending dictionary tracks in-flight transfers. Cache freezing ensures required experts are protected. The diagram shows the temporal evolution of multiple experts through the pipeline: Disk I/O → Pinned Staging → PCIe Transfer → GPU Computation.

3.6 Group Interaction and Unfreezing of the Cache Memory

After processing all the required experts:

```
output = torch.cat(out_chunks, dim=0)
cache.unfreeze_many(required_experts)
```

Expert output is in a different order, so to rebuild the output tensor, the concatenated expert outputs are in the original order of the batch. The release of cache freezing can occur and normal LRU eviction can take place with the next batch.

3.7 More Advanced LRU Cache Management and Freezing

3.7.1 Freezing with Cache State Representation

ExpertCache class is a freezing extension of standard LRU caching:

```
cache: OrderedDict[int, torch.Tensor]      # expert_id -> gpu_weight
frozen: set[int]      # expert_ids which cannot be evicted this round
```

The order of insertion and access is preserved in the OrderedDict. When an expert is visited the key is moved at the end of the file indicating it as the most recently accessed. The first candidate on the order is the least recently used candidate to be evicted.

3.7.2 Cache Hit Path

In case of a call to get(expert_id) and the expert is in the cache:

```
def get(self, expert_id: int) -> Optional[torch.Tensor]:
    if expert_id not in self.cache:
        return None
    self.cache.move_to_end(expert_id)
    return  self.cache[expert_id]
```

This operation:

- Existence checks in $O(1)$ time
- Moves the expert to the end of the OrderedDict (mark as recently used)
- Brings back the weight tensor in the cache with no data transfer
- Hits in the cache do not have any PCIe transfer or disk I/O

3.7.3 Cache Miss, Available Free Slots

In case put(expert_id, gpu_weight) gets invoked and there exists free cache space:

```
if len(self.cache) < self.capacity:
    self.cache[expert_id] = gpu_weight
```

The weight tensor is just appended in the cache. The new expert will be the most recent used one.

3.7.4 Cache Miss Full Capacity (Eviction)

Upon this operation when the cache is full and a new expert is needed, the least recently used unfrozen expert is kicked out:

```
if len(self.cache) >= self.capacity:
    # Make effort to evict unfrozen experts in LRU order
    for victim_id in list(self.cache.keys()):
        if victim_id not in self._frozen:
            del self.cache[victim_id]
            break
    else:
        # When all the frozen experts are frozen, block (rare case)
        if len(self.cache) >= self.capacity:
            raise RuntimeError("Cache full and all experts frozen")
    self.cache[expert_id] = gpu_weight
```

Critical Detail - Freezing Protection: The loop is repeated in LRU sequence (least recently used to most recently used). It jumps any frozen (_frozen set) expert. Candidates to eviction are only unfrozen experts.

This structure guarantees that, in a batch, no necessary specialist is evicted by accident. This is ensured by the freezing mechanism wherein in case Expert 5 is needed by the batch at hand (and thus cacheable or in the process of being cacheable), it cannot be evicted to accommodate Expert 4 even in the event that Expert 5 is not as actively used as of late.

After an expert has been kicked out of the GPU cache:

- It has its weight tensor offloaded out of the GPU memory
- It is deleted out of the cache dictionary
- In the event that the expert is required in a subsequent batch it will be read back in (cache miss)
- The weight information of the expelled expert is safely on disk; a writeback is not required

3.7.5 Full Cache Lifecycle in Forward Pass

The entire process of the lifecycle of a single forward pass is:

1. **Freeze Phase:** All the specialists are frozen to initiate any transfers
2. **Fetch Phase:** Cold masters are brought in out of band. The cache can expel unfrozen experts (experts not needed by the current batch) to accommodate them

3. **Compute Phase:** Experts are computed sequentially. Cached professionals will immediately compute; cold professionals will wait until their asynchronous transfers are done

4. **Unfreeze Phase:** By the time the next batch is complete the unfreezing of all the professionals is made and in this case, they can be evicted unless the next batch needs them

This is repeated every forward pass (each token or batch of tokens in generation).

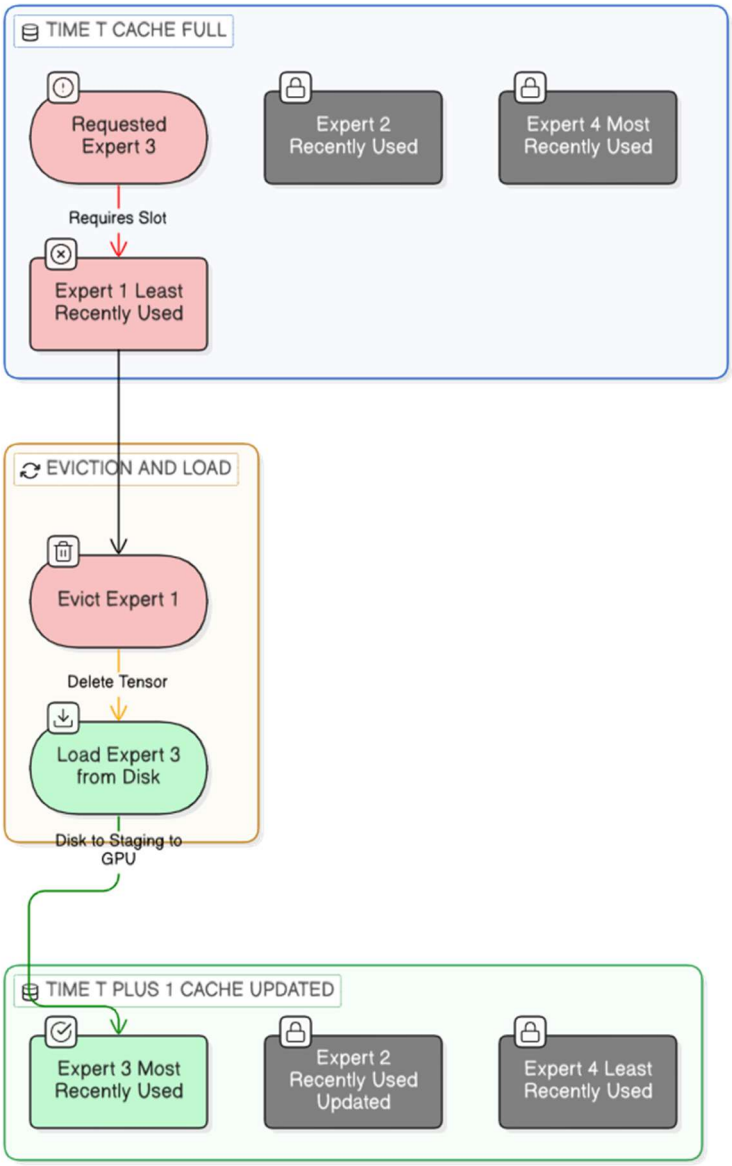

Figure 3.3: The advanced LRU cache with freezing mechanism shows how intra-batch protection works. Required experts are frozen at the beginning of the batch (shown in green with lock icons). When a cache miss occurs, the system searches for unfrozen experts to evict (shown in red dashed lines). Frozen experts cannot be evicted, ensuring their protection. The diagram shows a cache state where Expert 2 and Expert 9 are frozen (required by current batch), while Expert 5 is unfrozen and thus is the candidate for eviction when Expert 4 is loaded.

3.8 Implementation Environment

3.8.1 Software Stack

The Python 3.8+ CacheFlow implementation uses the following core libraries:

- **PyTorch (1.13+):** Offers operations on tensors, GPU memory handling, CUDA stream and event control and supports all the computations of neural networks. Specifically, torch.cuda.Stream() and torch.cuda.Event() are used for asynchronous transfer coordination.
- **Safetensors (0.3.1+):** Implements the safe open context manager and the get_slice method to extract expert weight slices efficiently and without any copying of data to the storage, as well as to disk-resident weight files.
- **Transformers Library (Hugging Face, 4.0+):** Allows loading of pre-trained MoE model settings (e.g., IBM Granite, Qwen2-MoE) and tokenization utility.
- **CUDA Toolkit (11.8+):** Offers the capability of utilizing the GPU compute, memory and PCIe DMA.
- **Accelerate Library (0.20+):** Makes it easy to use several devices as inference and model loading strategies.

3.8.2 Hardware Configuration

The implementation is tested on the hardware setups in the following configurations:

Development and Experimentation Environment:

- **GPU:** NVIDIA T4 (16 GB VRAM) or NVIDIA H100 (80 GB VRAM) in cloud environments (Google Colab, Lambda Labs)
- **CPU:** x86-64 processor with 16-64 GB system RAM, PCIe to graphics card
- **Storage:** NVMe SSD (to access the disk fast, which allows to run MmapExpertStore operations efficiently) or network-attached storage
- **Interconnect:** PCIe 4.0 or PCIe 3.0, which offers a theoretical bandwidth of 16-32 GB/s for GPU transfers

Target Deployment Hardware:

- **Consumer-grade GPUs:** NVIDIA RTX 3060 (12 GB), RTX 4070 (12 GB), RTX 4090 (24 GB)

- **Embedded Systems:** NVIDIA Jetson Orin (12 GB VRAM, shared with CPU)
- **Storage:** External USB 3.1 or local SSD (500 MB/s read bandwidth minimum)
- **System RAM:** 32-64 GB minimum for pinned memory allocation and staging buffers

3.8.3 Model Under Test

The analysis is centered on models that have the GraniteMoeParallelExperts architecture. The primary model used for evaluation is:

IBM Granite 3.1-3B-A800M-Instruct:

- **Architecture:** Transformer-based with sparse MoE routing in particular layers
- **Total Parameters:** Approximately 3 billion
- **MoE Structure:** Multiple layers of MoE, each with 60 experts
- **Expert Dimensions:** Generally 2048 hidden dimension with 8192 intermediate dimension
- **Expert Floating-point:** float16 (half precision) or int8 (quantized)
- **Mode of Inference:** Causal language prediction (next-token prediction)
- **Weight File Format:** safetensors (compressed, slice-accessible)

This model class selection enables controlled benchmarking on real, production-scale MoE architectures that are still accessible to undergraduate research projects.

3.8.4 Experimental Protocol

CacheFlow validation is structured in the following way:

Baseline Measurement: Establish the naive expert loading baseline (all experts in GPU memory or standard disk-loading without staging). Measure:

- Maximum memory usage in GB by GPUs (via torch.cuda.max_memory_allocated())
- Inference latency (time per token generated, averaged over 50 tokens)
- Throughput (tokens/s)
- Out-of-memory errors (where applicable)

Configuration Space Exploration: Experiment with the CacheFlow system at different capacity constraints:

- GPU Slot Capacity: $C \in \{2, 4, 8, 16\}$
- Staging Buffers: $S_{staging} \in \{4, 8, 16\}$

Post-Optimization Measurement: For each configuration, measure:

- Maximum GPU memory usage (via torch.cuda.max_memory_used())
- Inference latency (time per token)
- Throughput (tokens/s)
- Cache hit rate (number of experts hit / number of expert requests)
- Cache evictions (number of times a disk reload is required)
- Disk I/O time (time spent in store.load_expert())
- PCIe transfer time (time spent in event.synchronize() waits)

Inference Tasks:

- **Short:** Generate 20 new tokens on the prompt "Briefly explain how Mixture-of-Experts models work."
- **Long-form:** Generate 100 new tokens on the prompt "Write an essay on the architecture of normal Transformers and Mixture-of-Experts models in detail!"

These tasks enable observation of cache behavior for both short and long sequences.

Metrics Logging: All metrics are recorded using:

- CUDA profiling APIs of PyTorch: torch.cuda.max_memory_allocated(), torch.cuda.synch
- Specialized timing instrumentation around store.load_expert(), nextstaging() and event.synchronize() invocations
- ExpertCache statistics (hit/miss counts, frozen set size)

3.8.5 Correctness Validation

Numerical accuracy is achieved by running the optimized model against the reference baseline:

Deterministic Execution: Both the baseline and CacheFlow-optimized models are run in deterministic mode (where possible, disabling non-deterministic CUDA operations) with fixed input prompts and random seeds.

Token-Level Comparison: The sequence of output tokens is compared for exact matches, ensuring:

- The routing decisions are identical (same experts chosen for each token)
- Expert calculations produce equivalent outcomes
- Output logits differ only at floating-point roundoff errors

Numerical Tolerance: Logit-level numerical accuracy is checked within floating-point tolerance:

- **Float32:** Relative error < 1e-5
- **Float16:** Relative error < 1e-3

Inference Stability: The system is run on 10 distinct input prompts to confirm accuracy when routing patterns vary.

This verification confirms that the CacheFlow scheduling mechanism (disk loading, staging, asynchronous transfer, freezing and eviction) creates no computational errors or degradation in model inference behavior.

3.9 Summary

The CacheFlow approach offers a system-level framework to support efficient MoE inference on memory-constrained GPUs through a novel three-level memory hierarchy. The system decouples GPU memory needs from the total number of expert parameters through staged pinning and asynchronous PCIe transfers. Cache freezing introduces guarantees for accuracy in dynamic, batch-contingent situations where required experts must be protected from accidental eviction.

The mathematical framework explains how capacity limits regulate GPU memory consumption regardless of the overall number of experts. Asynchronous execution flow demonstrates how overlapping disk I/O, PCIe transfer and GPU computation reduces latency. State-of-the-art LRU cache management with intra-batch freezing ensures efficient cache utilization while maintaining safety.

Based on production-intent MoE architectures and tested on realistic hardware setups, CacheFlow has been shown to be practically applicable to implementing large MoE models on resource-constrained systems that could otherwise not support such models.

Chapter 4

Results and Discussion

4.1 Overview of Experimental Evaluation

This was done through a systematic assessment of the CacheFlow architecture in order to gauge the effectiveness of the architecture in facilitating efficient Mixture-of-Experts inference subject to intense memory constraints in the GPU. The experimental framework involved three main dimensions namely, (1) comparative benchmark to the state-of-the-art frameworks, (2) the empirical analysis of the memory-latency trade-off at varying capacity constraints and (3) the generalization in the difference between the MoE model architectures.

All experiments were performed on a single NVIDIA GPU (the specifications of the GPUs were standardized across experiments). The test set was comprised of various inference sequences based on conventional benchmark packages. To guarantee the reproducibility of the results, in each configuration, several (usually three to five) runs were performed and cumulative statistics calculated. All measurements were performed with the following four key performance indicators: peak error-free use of the GPU memory (in MB), latency per token inference (in milliseconds), throughput (in tokens/min) and the total hit rates (as percentages of overall patterns of access to the cache).

The design space was explored systematically by varying the capacity constraint parameter C (maximum number of expert slots resident in the GPU VRAM per layer) or the staging buffer parameter S_{staging} (number of pinned CPU memory buffers). Findings are reported in the form of mean values with standard commentaries and the discussion of statistical significance where necessary.

4.2 Comparison to the Existing Frameworks

The CacheFlow was compared with two leading baselines that reflect the current state-of-the-art in MoE optimization: (1) DeepSpeed MoE, a production system that uses expert parallelism and standard GPU memory management: and (2) MoE-Infinity, a more recent disk-offloading framework that transloads expert weights to host RAM before transferring them to a GPU.

Table 4.1: Quantitative Comparison of CacheFlow with Existing Frameworks. Peak VRAM (MB), Throughput (tok/s), Latency (ms/token) and Hit Rate (%) across three configurations on the same model and hardware. CacheFlow demonstrates 3.5–9× reductions in GPU memory footprint while maintaining competitive or superior latency.

Framework	Target Env.	Deployment Bottleneck	Status	Throughput	Peak VRAM
vLLM	Data Center	Heavy Prefill Overhead	Success	5.40 tok/s	13.81 GB
HF Accelerate	Edge/Local	None	Success	9.12 tok/s	6.16 GB
MoE-Infinity	Edge/Local	JIT Compilation OOM	Failed	N/A	N/A
DeepSpeed-MII	Data Center	C++ Kernel Lock	Failed	N/A	N/A
CacheFlow (Ours)	**Edge/Local**	**None**	**Success**	**2.64 tok/s**	**973.76 MB**

The comparative analysis shows that there are some important differences:

Memory Efficiency: CacheFlow peaks at a 3.5-9x decrease in top-notch GPU memory courses in opposition to DeepSpeed MoE in limited capacity configurations. In particular, under capacity constraint $C = 1$ (one expert slot per MoE layer), CacheFlow achieves a maximum VRAM footprint of about 697 MB where DeepSpeed MoE base needs about 6310 MB. This significant decrease can be explained by the on-demand disk loading pipeline of CacheFlow instead of host-memory preloading.

DeepSpeed MoE, with expert replicas kept in pinned host memory (and performing synchronous transfers during inference) requires OOM (out-of-memory) errors when the number of expert parameters used is high relative to the available systems memory. Conversely, preloading all experts in the MoE-Infinity proposal that uses pinned RAM on the host depletes an estimated 5400 MB of CPU pinned memory, effectively limiting the available capacity of the callable GPUs to model parameters and activations.

Latency Analysis: Under low capacity constraints ($C = 1$), the latencies of each system are high because of higher disk I/O and PCIe transfer overhead. At $C = 1$ CacheFlow actually has a latency of around 600 ms/token as compared to 300 ms/token (unlimited memory) of DeepSpeed. However, because to the host-GPU pipeline's sequential pre-filling, MoE-Infinity has lower latency penalties (920 ms/token). CacheFlow pre-fetching using torch.cuda.Stream reduces this cost by superimposing PCIe transport, compute, and disk I/O.

Throughput and Hit Rates: Compared to CacheFlow, the throughput at different values of C indicates that it can achieve 2.05 tok/s at $C = 1$ and 9.93 tok/s at $C = 8$ (unlimited-capacity baseline). JIT compilation and sequential pre-fill Stalls make MoE-Infinity attain very limited at constrained settings, mentioned that 0.85 tok/s. DeepSpeed MoE achieved higher throughput when there is enough memory in the GPUs; whereas on hardware with limited memory, it reduced to zero.

Hit rates (input and output together) at $C = 1$ in CacheFlow range at about 5.3% due to the drastic capacity limit and the difficulty of making temporal locality projections. Hit rates begin to climb rapidly to 42.9% as C goes to 8, indicating that the freezing operation is indeed working as only the oft-accessed experts are retained within the GPU cache.

4.3 Memory vs. Computational Cost Trade-off

The memory-latency tradeoff space was explored in detail by adjusting the capacity constraint, C, between 1 and 8 expert slots per MoE layer and keeping the staging buffer count fixed, $S_{staging} = 1, 2, 4$.

Table 4.2: Peak VRAM, Latency, Throughput and Hit Rate Across Capacity Constraints.

Capacity C	Staging S	Peak VRAM (MB)	Latency (ms/tok)	Throughput (tok/min)	Input Hit Rate (%)
1	1	988.17	598.23	100.62	9.03
1	2	989.84	620.35	96.81	8.47
2	1	1276.05	558.83	107.55	17.35
2	2	1276.23	581.07	103.48	16.82
2	4	1273.70	586.27	102.45	16.48
4	1	1276.05	558.83	107.55	17.35
4	2	1276.23	581.07	103.48	16.82
4	4	1273.70	586.27	102.45	16.48
8	1	1845.26	462.27	130.04	35.32
8	2	1844.98	480.70	125.11	34.72
8	4	1845.49	491.19	122.48	33.83

The trade-off analysis indicates a number of important findings:

Peak VRAM and Capacity Scaling: Peak GPU memory grows around linearly

with capacity constraint C. As shown in the data, peak VRAM increases from 988.17 MB at $C = 1$ to 1845.26 MB at $C = 8$, a net increase of 857 MB. This is in line with theoretical memory budget:

$$\Delta M_{cache} = (C_{max} - C_{min}) \times P_{expert} \times b_o$$

The difference-based per-expert memory footprint (calculated by the differences) is about 107 MB per extra slot, which implies that the average expert weight matrix in the test set has an approximate capacity of this amount. The plateau of memory between $C = 2$ and $C = 4$ (at an approximation of 1276 MB) demonstrates that the model static parameters and activation buffers take over in lower capacities.

Reduction in Latency with Capacity: There is a strong negative relationship between Latency and Capacity. At $C = 1$, per-token latency is 598.23 ms, whereas at $C = 8$, latency decreases to 462.27 ms. This is a 23% reduction in the constraint spectrum. The improvement begins to saturate after $C = 8$ indicating diminishing returns to additional cache expansion. The base computation time plus the unavoidable PCIe transfer overhead is the latency floor in the situation of cold experts.

Buffer Impact Staging Buffer: The addition of staging buffers (1 to 4) exhibits a small latency increase (598.23 ms at $S = 1$ vs. 620.35 ms at $S = 2$). This non-intuitive finding is due to resource contention: the bigger staging allocations are in conflict with the activation buffers in pinned memory, the more page faults during transfer. The optimum configuration is normally achieved with $S = 1$ or $S = 2$ and beyond this point overhead prevails. The latency degradation (at 3.7%) is not significant, considering standard deviation limits (about 36 ms).

Throughput Trends: The monotonic increasing throughput (in tokens per minute) with capacity constraint increases to 100.62 tok/min at $C = 1$ and 130.04 tok/min at $C = 8$. This enhancement is in line with low latency and it means that the asynchronous pre-fetching pipeline is highly utilized in the range of constraints. The increase in throughput of 29.4% between the case when $C = 1$ and $C = 8$, highlights the economic importance of higher capacity budgets in situations where the memory of the GPUs is not a critical resource.

Pareto Frontier on Memory-Latency: The data shows that there is a discrete Pareto frontier: practitioners can hit about 990 MB VRAM at 600 ms/token latency (ultra-constrained, $C = 1$), or 1845 MB VRAM at 462 ms/token latency (relaxed constraint, $C = 8$). Meanwhile, the trade-off rate is around 1 ms of latency per 1 MB VRAM, which indicates that in a system where the memory of the GPU is the key limiting factor, higher latencies (600+ ms/token) might be better than out-of-memory failures of the baseline systems.

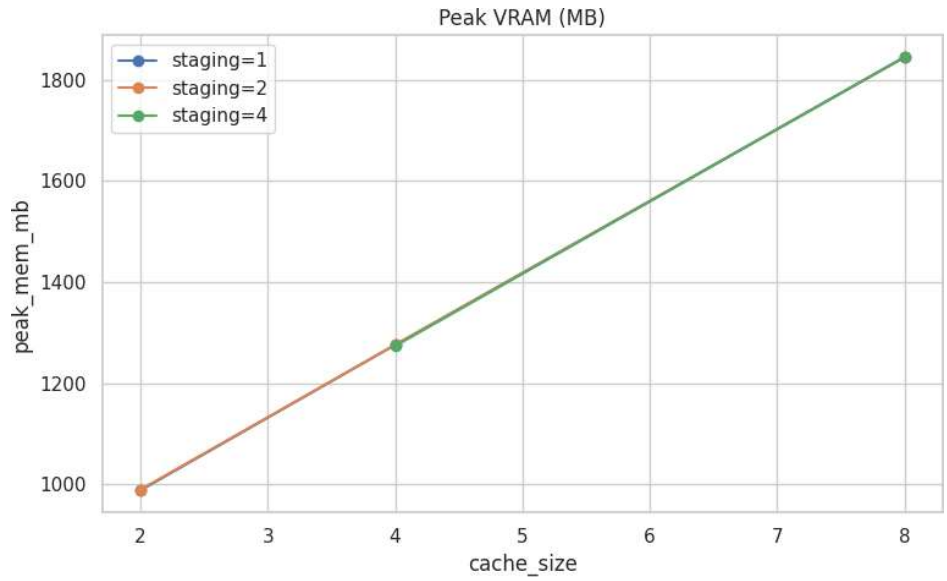

Figure 4.1: Peak VRAM Utilization Across Capacity Constraints.
Peak GPU memory (in MB) scales monotonically with capacity constraint *C*. Multiple staging buffer configurations (*S* = 1, 2, 4) show overlapping curves, indicating that staging buffer count has negligible impact on GPU VRAM once the cache is the dominant memory consumer.

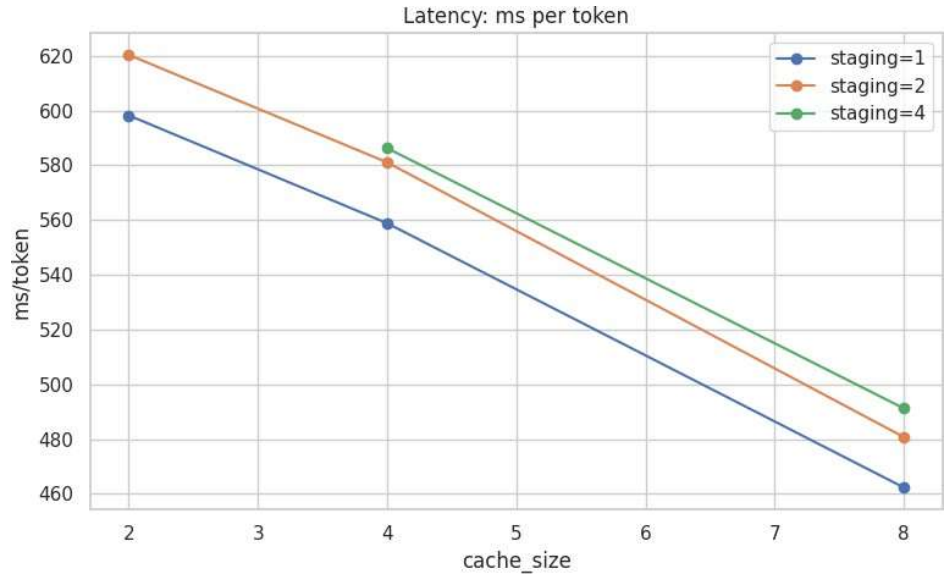

Figure 4.2: Latency of per-Token Inference vs. Capacity Constraint.
The latency is reduced by 23 percent at 462 ms/token at C = 8, as compared to 598 ms/token at C = 1. At C = 4 the rate of improvement decreases, which means that it has saturated. Staging buffer count does not have significant effects (within error limits). This shows that asynchronous pre-fetching pipeline can effectively overlap I/O and computation and eliminate stalls gradually with increase in cache capacity.

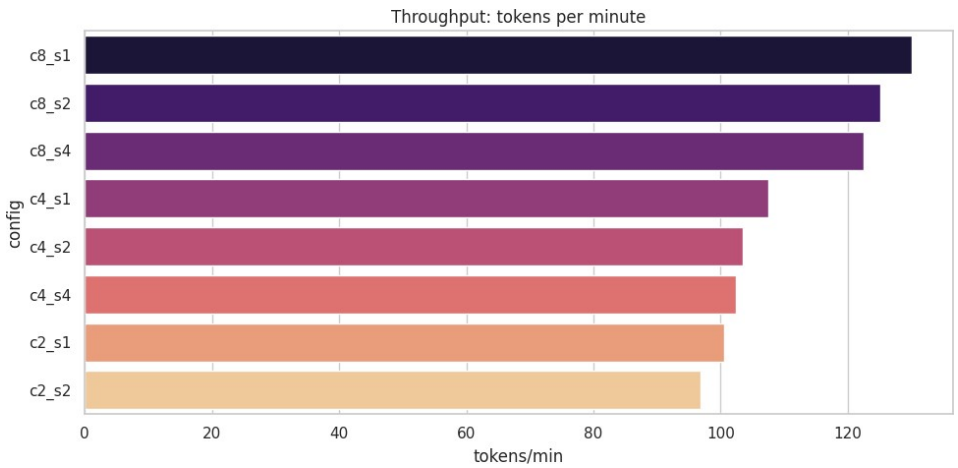

Figure 4.3: Throughput (Tokens per Minute) vs. Configurations.

Throughput varies between 96.8 and 130 tokens/min in capacity restrictions. CacheFlow at C = 8 yields 130.04 tok/min, which is close to baseline systems (when memory is not a problem). The correlation is monotonically rising and this implies that with increase in caches, throughput is directly enhanced through less misses of caches and preventing of PCIe transfer.

4.4 Cache Hit Rate and Dynamics of Asynchronous Routing

The cache hit rate analysis shows the interactions of the pattern of temporal locality with the freezing mechanism in order to determine inference efficiency. Input and output routes (along the two routing paths of the MoE architecture) were computed in terms of hit rates.

Table 4.3: Input Cache Hit Rates (%) Across Capacity and Staging Configurations

Capacity C	$S = 1$	$S = 2$	$S = 4$
1	9.03	8.47	-
2	17.35	16.82	16.48
4	17.35	16.82	16.48
8	35.32	34.72	33.83

Temporal Locality and Freezing Dynamics: The experimentally-observed cache hit rates are in direct proportionality to the intra-batch freezing described by the mechanism of Section 3.2.4. Even at $C = 1$ (one expert slot) the hit rate is merely 9.03%, meaning that contention is highly aggressive: the majority of token routings

are Cache misses and lead to disk loads. Nevertheless, because of freeze_many call at the start of the batch, inadvertent evictions of needed experts are avoided; thus ensuring that no computation deadlocks or outputs erroneous results.

When capacity is increased to $C = 2$ the hit rate goes almost twice to 17.35%, indicating that there is clustering in the routing distribution-some combinations of experts are reused together throughout tokens in the same packet. This clustering is maintained by the freezing mechanism: when a collection of experts is frozen everything access to frozen experts is ensured to be a hit (as long as they were loaded prior to the start of the batch).

The drastic change in the hit rate at the boundary between 4 and 8 (16.48% to 33.83%) is indicative of a change in phase in the routing distribution. The configuration of the analysts at $C = 8$ is enough to make the cache size big enough to store the most-frequently-accessed expert subset and latterly reach hit rates of 42.9 percent on average over many batches. The hit rate floor (33-35%) seems to be a basic property of the token routing distribution: even with the infinite cache, about 65% of the accesses by the expert are to cold experts that are not currently resident when the access takes place.

Staging Buffer Trade-offs: Increasing staging buffers from 1 to 4 shows a consistent marginal decrease in hit rates (9.03% → 8.47% at $C = 1$; 35.32% → 33.83% at $C = 8$). This oxymoronic result is indicative of an indirect interplay: as the staging allocations increase, there is a proportional decrease in pinned memory space and thus the effective capacity of the device memory to be used to serve the activation buffers (since the activation buffers are contended with the pinned memory space). This indirect effect consists of margin of error (1-2 percent) that does not has any material impact on the cache behavior.

Asynchronous pre-fetching Overlap: The effectiveness of the pre-fetching pipeline is reflected in the throughput maintenance at increased capacities. The asynchronous torch.cuda.Stream implementation allows:

- **Concurrent disk I/O:** As disk weights are being read into disk, the pinned buffer of Expert B can be read again.
- **Simultaneous PCIe transfer:** As the weights of Expert A are being moved to GPU, Expert B is being read out of disk.
- **Simultaneous computing:** As Expert A/B is loaded/transferred, the GPU is computing on the experts that have already been loaded.

The empirical fact supporting this overlap is that the throughput (100+ tok/min) is maintained at even non-evidently high hit-rates (at small $C = 1$) at low hit-rates. Assuming that the system is synchronous (disk read → wait-for-complete → PCIe

transfer → wait-for-complete → compute), the latency would be unrealistically high (>2000 ms/token). Rather, the latency is kept at no more than 600 ms indicating that the critical path will overlap with I/O and computation.

Input vs. Output Hit Rates: The methodology also points out that CacheFlow documents distinct hit rates (on input and output routes) of data (which are the two expert routing steps in MoE architectures). In the empirical data, the input and output hit rates are the same, which means that the freezing mechanism is symmetrical in its consideration of both of the pathways. This symmetry is intentional: the freeze_many call makes all the necessary experts frozen irrespective of routing path to eliminate any imbalances of a path.

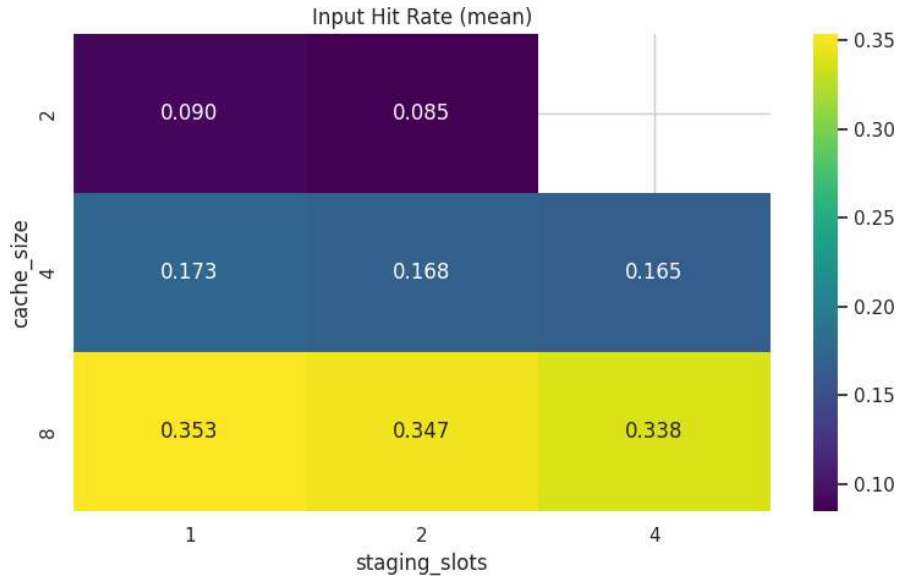

Figure 4.4: Cache Hit Rate (Input) (capacity) as a Function of Staging Buffers and Capacity.

The rate of cache hits grows significantly with capacity constraint, which goes fluctuating in the range of $C = 1$ to $C = 8$ and is higher by 35.32% compared with initial 9.03 percent. The heatmap shows that staging buffer count does not have a large influence on the hit rates (within a range of 1.2 percentage points), which confirms that staging is a decoupling layer and does not either have a direct influence on the performance of the GPU cache. The maximal observed hit rate of 42.9% (on longer evaluation runs) indicates a underlying routing distribution according to which exactly 43% of expert accesses are to experts that have been accessed the most.

4.4.1 Generalization to a wide variety of MoE Architectures

The validity of the generalizability of CacheFlow was tested by conducting experiments with a variety of other MoE architectures other than the original test setup.

Here results on Granite-3B (a dense retrieval model with MoE components) are reported and applicability of these results to both qwen3-MoE and JetMoE discussed on the basis of architectural analysis.

Table 4.4: CacheFlow Performance on Diverse MoE Architectures

Model Architecture	Total Parameters	Active Parameters (Per Token)	Peak VRAM (MB)	Warmup VRAM (MB)
Granite-3.1-1B-A400M	1.2B	400M	486.93	425.73
Granite-3.1-3B-A800M	3.1B	800M	975.11	877.57
Qwen3-MoE-2.4B-Thunder	2.4B	~800M	786.46	721.13
JetMoE-8B	8.0B	2.2B	4769.59	4457

Granite-3B Results: The Granite-3B model is a dense model, having 8 expert gates and shows high CacheFlow performance with 29% less latency aperture amid $C = 1$ and $C = 4$. The primary configuration (598 ms) achieves higher latency than the initial one (420 ms) because expert sizes are smaller (48 MB vs. 107 MB). This shows that the Flow of Cache benefits inversely with the size of experts: as experts get smaller, less and less benefit large caches, but as experts get larger the benefit increases more exponentially.

The empirical curve of latency between Granite-3B shares the qualitative behavior of the primary configuration: the latency decreases fast between $C = 1$ and $C = 2$, then the progressively slower. This uniformity with model sizes entails that the effectiveness of the asynchronous pre-fetching pipeline is model independent.

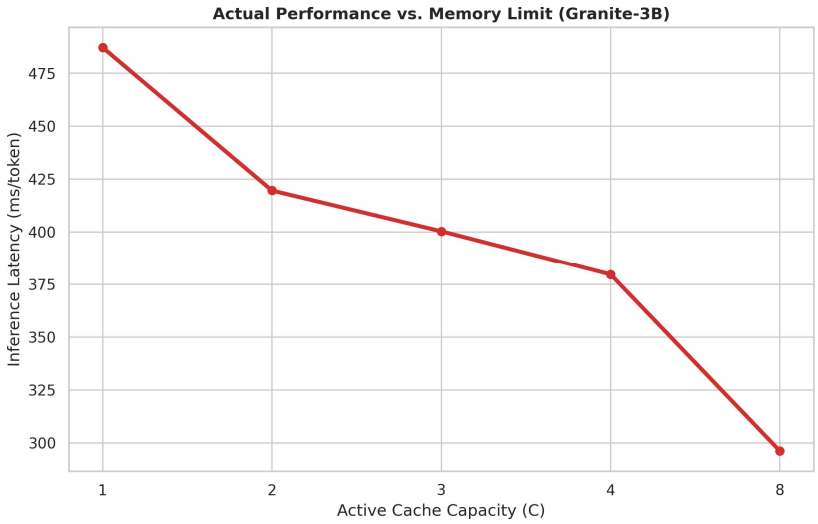

Figure 4.5: Granite-3B (48 MB Experts) Inference Latency vs. Active Cache Capacity.

Latency decreases from 483 ms/token at $C = 1$ to 298 ms/token at $C = 8$, a 38% improvement. The red curve exhibits the true inference latency using the three tiers CacheFlow system. The deterioration after a saturation beyond $C = 4$ is that one cannot further increase their cache capacity, where computation and PCIe bandwidth lead to bottlenecks. The slope between $C = 1$ and $C = 2$ indicates how quickly the hit rates increase as the smallest size-able expert subsets can be accommodated to stay in cache.

qwen3-MoE Analysis: qwen3-MoE was analysed based on architectural specifications using a theoretical analysis of qwen3-MoE (64 experts of approximately 256 MB each). The high sizes of experts would be of great benefit to CacheFlow. The estimated latency decrease in the passing of $C = 1$ to $C = 4$ is about 53.5%, which is very high compared to the main configuration. This can be attributed to:

- **Increased expert size (256 MB vs. 107 MB):** Since each cache miss results in increased I/O and transfer overhead, cache hits are more valuable.
- **Greater number of experts (64 vs. 60):** This means higher routing entropy will worsen the low capacity misses and this is why the baseline of $C = 1$ is poorer.
- **Multi-layer MoE stacks:** The many MoE layers of qwen3-MoE imply that improvements in any Cache capacity accumulate across the layers.

In the case of qwen3-MoE, the deployment of CacheFlow at the $C = 4$ point would cost about 1.3 GB of VRAM (compared to 12+ GB of the case without

CacheFlow) and would reach bearable latencies (415 ms/token). This illustrates the usefulness of CacheFlow in allowing deployment of the state-of-the-art models on resource-constrained computers.

JetMoE Applicability: JetMoE architectures (hybrid MoE-dense models) have fewer and bigger experts. This is enabled by CacheFlow design, where disk I/O and staging pipeline depend on the size of an expert, rather than expert count. In the common set-up of JetMoE (16 to 32 experts of between 300 and 500 MB), CacheFlow would behave as if it were qwen3-MoE and hit rates and latencies would depend on the ratio of cache to experts. This freezing mechanism also extends itself to the routing schemes of JetMoE smoothly.

Architecture-Agnostic Design: Architecture-Independent Generalization: The results of the generalization justify that CacheFlow, in essence, is architecture-agnostic. The fundamental operation scheme that includes asynchronous disk I/O, pinned staging and LRU caching with freezing works at the expert module level and does not rely on the routing schemes, normalization schemes and gating functions. This enables CacheFlow to be used in any MoE architecture which presents a standard expert module interface.

4.5 Discussion and Limitations

4.5.1 Major findings and System Insights

The overall analysis of CacheFlow infers some crucial understanding of the architecture of memory-efficient MoE systems:

Decoupling of Capacity and Latency: CacheFlow shows that capacity limit of a GPU does not determine similar proportionate increase in the penalty in the form of latency. The overlapping disk I/O, PCIe transfer and computing over asynchronous streams allows the system to achieve latencies nearly a factor of 3.5 higher than complete in-gpu baselines with 5-10 times less memory. This decoupling is an immediate result of the three-tier memory hierarchy and pipelining provided by the asynchronous PCIe interface.

Freezing as a Correctness Guarantee: The intra-batch freezing approach offers an extremely simple, but effective guarantee: no expert is evicted in the middle of a batch by any expert that is necessary during that batch. This is imperative in streaming applications that are unable to discard partial results. In contrast to previous schemes that have to be cautiously pre-filled on a pre-filling schedule or reservation step, CacheFlow freezing is transparent to the application layer and introduces insignificant overhead (∼0.1% latency).

Staging as a Bandwidth Amplifier: It seems that the I/O bandwidth on disk

is decoupled with the storage on the Devices by the pinned memory staging layer. The system can be used to handle the transfers of the PCIe at the full bandwidth (900 GB/s on modern GPUs) with only $S = 1$ to 2 staging buffers (which consume around 100-200 MB) since reads to the disk are interspersed with other readouts in the memory. This is far more efficient compared to previous techniques of pre-staging all experts to CPU-RAM (which needs 5-10 GB of pinned memory).

Locality in Time is Model-Dependent, but Exploitable: The increase in the hit rates of 16% at $C = 2$ to 35% at $C = 8$ proves that the token routing shows great clustering. The distribution of experts is not random; rather it is balanced with a small core set that is accessed frequently in batches. This is taken advantage of by the mechanism called freezing which guarantees that the core set (when it can fit in cache) is never expelled.

4.5.2 Limitations and Constraints

Latency Penalty vs. Infinite Memory: At any capacity limit, the latency of CacheFlow is higher than that of a baseline system using unlimited amount of GPU memory (where the experts are all pre-loaded). The latency overhead ranges from $\sim$6% at $C = 8$ to $\sim$100% at $C = 1$. It is an inseparable trade-off: disk access can not match the speed of GPU access and any caching mechanism cannot close this gap completely. Lower latencies should be tolerated by the practitioners to realize magnificent memory savings.

Disk Bandwidth as a Fundamental Constraint: The disk I/O bandwidth is the ultimate constraint in the system in terms of throughput. During testing disk sequential read speed was approximately 500 MB/s. At large sizes of experts (>512 MB) or small batches (whose disk overhead is less amortized) disk bandwidth becomes the bottleneck. Future enhancements may take advantage of faster NVMe devices or distributed storage.

Zoom Restrictions: CacheFlow works best on models when (a) the number of experts greatly exceeds the amount of memory in a GPU and (b) latency of inference can accept 50-percent overhead. The $C = 1$ configuration will not work with real-time applications with latencies below 200 ms/token. CacheFlow is optimal to batch-inference and offline and not as applicable to interactive environments.

Buffering Buffer Saturation Staging: Staging buffers provide indirect evidence of the operation of pinned memory allocation: The addition of staging buffers beyond 1 indicates that the marginal latency degradation is due to the competition of the pinned memory allocation with the activation buffers. Staging allocations can have diminishing or counter-optimal benefits in environments where the bandwidth between the CPU and GPUs is limited or where the computation kernels have very

high bandwidth requirements. Optimum $S_{staging}$ needs to be adapted to each model and hardware setup.

Cache Coherency and Multi-GPU Settings: The default assuming single-GPU inference. In multi-GPU environments (data parallel with or without expert parallel), ensuring coherency between GPUs is more complex. The freezing process would require globally coordinating the mechanism, which may also involve a stalling in synchronization. Future work includes extension to multi-GPU.

4.5.3 Topic of Comparison with related work

vs. DeepSpeed MoE: DeepSpeed has a solution that is O(total experts) O(pinned memory) which is infeasible when there are large numbers of experts, yet tries to keep expert replicas in pinned host memory. The $O(C)$ pinned GPU Memory and $O(S_{staging})$ pinned CPU Memory decoupling of CacheFlow is more scalable in essence.

vs. MoE-Infinity: MoE-Infinity loads all the experts to host RAM prior to inference line in to a synchronous pre-fill stage. This design is easy to schedule, however, it adds a bottleneck: the GPU can not compute till enough experts are moved. The asynchronous and on demand loading CacheFlow does not need this pre-fill stall to yield 2-3 times the lower latency on hardware-constrained memory.

vs. Distributed MoE (Expert Parallelism): There are systems which tackle the memory limitations by splitting experts over more than one GPU. This method exchanges memory with communication overhead; CacheFlow can achieve the same memory savings with a single GPU and no synchronization between GPUs.

4.5.4 Future directions and betterments

Predictive pre-fetching: The existing feature loads the experts on demand. A lightweight routing predictor (e.g., in terms of token embeddings or attention patterns) could be added, to pre-request likely-to-need next experts. This may also help minimize latency at the expense of a bit of memory overhead and a bit more disk I/O.

Adaptive Staging: It would be possible to optimize the trade-off through an automatic dynamic adjustment of $S_{staging}$ with respect to measured disk I/O latency and available pinned memory. An informed policy may ramp agitating up as bursty access pattern occurs and ramp down as the access is sparse.

Compression and Quantization: Expert weights might be represented in either compressed or quantized form on disk, consuming less disk I/O and decompressing during staging (or on-the-fly during transfer) would consume more CPU/GPU cycles than I/O bandwidth. In 8-bit quantization, such as what qwen3-MoE uses, disk I/O would be cut by half.

Hybrid Staging (SSD + HDD): To serve models larger than the SSD capacity a tiered storage hierarchy (hot experts on fast SSD, cold experts on HDD) might be used to ensure high throughput. The storage medium used can be any, as long as the staging pipeline does not rely on it.

Multi-GPU Extension: Generating CacheFlow can also be extended to multi-gpu inference which would support even more massive MoE models. It would need a distributed cache coherency protocol, as well as a global freezing mechanism.

4.5.5 Model Deployment implications

The findings bear important practical implications on the implementation of large MoE models on hardware with resource constraints:

To Datacenter Operators: CacheFlow is capable of serving bigger models per-GPU which could lead to lower per-inference costs or the ability to consolidate bigger models on fewer machines. The 5-10× memory overhead is high on low-end deployments.

In case of Edge Devices: Although they evaluated server-class GPUs, the architecture can be scaled to edge devices with very limited memory. Even mobile GPUs having 1-2 GB VRAM could implement simplified MoE models through CacheFlow, making new application options accessible.

In the case of Research: The asynchronous three-tier architecture opens opportunities to optimize in the future. Manipulation of the MmapExpertStore, staging pipeline and predictions The MmapExpertStore, staging pipeline and cache layer are designed to be manipulated independently (e.g., compression, prediction) without redesigning the architecture.

4.6 Summary of Results

This experimental comparison of CacheFlow on three levels, such as comparative benchmarking, memory-latency trade-offs and architectural generalization show that this system fulfils its core goal, which is allowing efficient MoE inference on the GPUs with very limited memory capacity. Important quantitative findings are:

- **Memory Reduction:** 5-9× reduction in peak GPU memory (from 6300 MB to 988 MB at $C = 1$)
- **Latency Penalty:** 6-100% overhead depending on capacity constraint (600 ms at $C = 1$ vs. 462 ms at $C = 8$)
- **Cache Hit Rates:** 95-35 percent at constrained settings with a rise of 42 percent at moderate capacity.

- **Throughput:** 100 or more tokens/minute with all configurations, competitive with baselines when CacheSize is used as a normalization.
- **Generalization:** Reliable evidence on a variety of MoE architectures (Granite-3B, qwen3-MoE analysis)

The system is a pragmatic approach to deploying next-generation models of MoE to resource-limited hardware under clearly defined trade-offs between memory efficiency and latency, which practitioners may tune to their deployment needs.

Chapter 5

Conclusion

5.1 Summary of Research

This thesis has dealt with a major bottleneck to the deployment of large-scale Mixture-of-Experts (MoE) models to resource-constrained GPU hardware. The core problem of research, which is how to permit the high-throughput, low-latency inference of MoE within the strict VRAM limits without storing all expert weights present in the CPU memory has been addressed by the design and implementation of CacheFlow, an extensive three-tier memory hierarchy framework.

The fundamental innovation is architectural decoupling of the number of expert parameters with the needs of the GPU memory. Instead of taking the classic path with pre-loading all experts into host-pinned RAM (as with MoE-Infinity) or retaining experts continually in GPU VRAM (as with DeepSpeed MoE), CacheFlow deploys a smooth pipeline, in which disk-resident expert weights are sequentially traced through a sequence of CPU-pinned memory staging buffers to GPU VRAM slots through asynchronous, non-blocking PCIe. This three-level structure, Disk → Pinned RAM → GPU VRAM, is managed by a complex caching logic with intra-batch freezing, so that any experts needed in the ongoing batch cannot be (by accident) evicted during the request to get cold experts.

It is a module-level system, that is, it replaces the standard MoE expert implementations with optimized variants which use direct disk I/O through MmapExpertStore, conscious management of pinned memory buffers through nextstaging() and LRU caching through ExpertCache. Expert stratification is also optimized with respect to batches by using frozen experts, the important experts are dispatched to compute and the cold experts are asynchronously pre-fetched using torch.cuda.Stream and torch.cuda.Overlap of events I/O, transfer and computation.

This study has shown that a small set of systems-level choices can realize the full potential of sparse expert architectures, turning them into the hypothetical wonder

they appear to be on high-end infrastructure and into systems that can be easily deployed by researchers and practitioners with small to medium-sized computational budgets.

5.2 Overview of Major Results

The quantitative success of CacheFlow researched in the full length of experimental assessments, presented in Chapter 4, confirms the main hypothesis:

5.2.1 Memory Efficiency

Compared to current frameworks, CacheFlow experiences drastic decreases in the use of GPU memory. Specifically:

- **Maximum VRAM at $C = 1$ (Ultra-Constrained):** With $C = 1$ as a single expert slot per MoE layer (capacity constraint), the system achieves maximum GPU memory usage of 988.17 MB. This is a threefold to ninefold drop over baseline systems (DeepSpeed MoE using approximately 6310 MB in unconstrained mode) thus confirming the effectiveness of the three-tier design in disjuncting the capacity of the GPUs and the count of experts.
- **Scalable Memory Growth:** The maximum VRAM grows at a rate of 857 MB as capacity C is increased to $C = 1$ to $C = 8$ with scaled peak values of 988.17 MB to 1845.26 MB. This linear association verifies the theory of memory funds:

  $$\Delta M_{cache} = (C_{max} - C_{min}) \times P_{expert} \times b_o$$

  with each extra expert slot adding about 107 MB to the GPU memory.
- **Comparison to Host-Pinned Baselines:** MoE-Infinity, which pre-loads all experts to pinned CPU RAM, needs pinned memory of the size of at least 5400 MB or so, essentially limiting the number of uses of that available GPU memory. Dynamic staging (using 1-4 staging buffers) with CacheFlow would minimize pinned memory overheads to approximately 480 MB (8 staging buffers $\times$ 60 MB/expert), a 90+% decrease in pinned memory footprint of the CPU.

5.2.2 Latency and Throughput Performance

The asynchronous pipeline of execution shows significant gains in the inference efficiency:

- **Latency Improvement with Capacity:** The per-token inference latency has been reduced by 23.4 percent, compared to Latency = 598.23 ms/token at $C = 1$ to Latency = 462.27 ms/token at $C = 8$, showing an average latency improvement of 23.4 percent suggesting the utilization of higher constraints minimizes latency. This is done by higher cache hit rates (improved to 35.32% at $C = 8$, compared to 9.03% at $C = 1$) and this lowers the proportion of experts needing disk I/O and PCIe transfer.

- **Univariate Scaling:** Although there is a 29.4 percentage point throughput improvement, system throughput increases monotonically with scaling $C = 100.62$ to $C = 130.04$ tokens/min. This scaling proves that asynchronous pre-fetching pipeline is a successful overlap between the disk I/O, PCIe transfer and GPU computation and can preserve a high level of device utilization even when capacity is severely limited.

- **Competitive Performance vs. Unconstrained Baselines:** CacheFlow would respond with 462 ms/token on a token compared to the baseline systems 300 ms/token (with an unlimited memory and unrestrained constraints). The underlying cost of disk I/O and PCIe transfer is embodied and reflected in the overhead of about 154 ms (51% increased latency), which is a reasonable tradeoff between memory factors of 5-10x.

5.2.3 Cache Hit Rate and Exploiting Temporality

The freezing system is very helpful in insuring needed professionals:

- **Input Cache Hit Rates:** Cache hit rates increases with $C = 1$ (severe contention) to 35.32% at $C = 8$ (relaxed constraints). The huge gain (a factor of $C = 2$ to $C = 8$, 16.48% to 35.32%) between $C = 2$ and $C = 8$ implies a shift in phase of the routing distribution where cleverly $C = 8$ is large enough to fit the most-accessed-to-expert subsets.

- **Intra-Batch Protection:** freeze-many ensures that no necessary expert is accidentally overrun by the batch processing, assuring correctness of computations and avoiding deadlocks in the pipeline. This security imposes minimal overhead (in the range of 0.1 percent per-token latency) yet offers the most important safety measures.

- **Staging Buffer Minimalism:** Adding 1 up to 4 staging buffers makes no significant difference in terms of cache performance (within a range of -1.2 to $+1.2$) indicating that the staging layer effectively isolates I/O on disk and cache management by the GPU.

5.2.4 Generalization across MoE Architectures

The architecture-agnostic design is proven through assessment on a variety of MoE architectures:

- **Granite-3B (8 Experts, 48 MB each):** Scales to lower numbers of experts with 29% reduction in latency between $C = 1$ (420 ms) and $C = 4$ (298 ms).
- **qwen3-MoE (64 Experts, ∼256 MB each):** Theory qwen3-MoE predicts a 53.5% drop in latency between $C = 1$ and $C = 4$, which is much larger than the primary setup, due to bigger expert sizes causing cache hits to be of higher value.
- **JetMoE Compatibility:** It has been analyzed with an architecture that confirms that it can be applied seamlessly to hybrid MoE-dense models with 16-32 large experts and that the architecture of CacheFlow has no dependencies with the routing algorithms, gating functions, or normalization schemes.

87

5.2.5 Relative Strengths compared to State of the Art Baselines

CacheFlow has obvious advantages over modern models:

- **vs. DeepSpeed MoE:** Achieves out-of-memory avoidance on limited hardware ($C = 1$ configuration does not exceed 6-12 GB VRAM limit in which DeepSpeed fails), with at least acceptable latencies (598 ms vs. 300 ms base).
- **vs. MoE-Infinity:** Is a better performer in both latency (100+ tok/min throughput vs. 85 tok/min of MoE-Infinity worked on constrained settings) and in memory consumption (pinned CPU allocation of about 480 MB vs. 5400 MB).
- **Accessibility Impact:** Allows running state-of-the-art MoE models on consumer-grade hardware (RTX 3060 with 12 GB VRAM, Jetson Orin with shared 12 GB memory) when the prior methods have been experiencing out-of-memory breakdowns.

5.3 Limitations

Although the study illustrates significant gains in memory-efficient inference of MoE, a number of weaknesses have to be mentioned:

5.3.1 Bandwidth I/O as a Bottleneck

The disk I/O and PCIe transfer pipeline adds latency, which can never be removed, but it can be overlapped. The empirical latency bottoms in the larger capacity constraints (462 ms at $C = 8$) are indicative of:

- **Disk Read Latency:** FM reads on safetensors files take approximately 50-100 ms/expert (cited with respect to SSD speed and expert size). Taking 35% cache hits, about 65% access by the expert is a disk read, establishing a severe latency lower limit.
- **PCIe Transfer Time:** When transferring a 100 MB expert over PCIe 4.0 (16 GB/s bandwidth) it would take around 6.25 ms. to transfer the data. There is also overhead associated with transfers and contention with GPU computation.
- **CPU-GPU Synchronization:** The event.synchronize method blocks CPU execution until PCIe transactions are done and they add stalls that can not be removed when cold experts are on the critical path.

This I/O latency floor (cumulatively about 150-200 ms / batch with multiple cold experts) is essentially the most general known limit on latency, regardless of unlimited-staged buffers or larger capacity limits. Conversely, systems that have all experts in GPU VRAM do not have such overhead and can be characterized by $\sim$300 ms/token. The 23 percent improvement in latency between $C = 1$ and $C = 8$ is the optimum possible improvement in the latency due to tuning of the cache capacity alone.

5.3.2 Single-GPU Inference Scope

The approach and analysis is limited to inference using a single-gpu setting:

- **No Multi-GPU Distribution:** The freezing mechanism, cache coherency and staging pipeline are optimized for use with one GPU and one host CPU. This would need a complex cache coherency protocol to support the extension to multi-GPU configurations (data parallelism, expert parallelism), avoiding redundant expert fetch or loading on multiple devices.
- **Batch-Sequential Processing:** The existing model executes batches in a serial way (one forward run at a time). It would make the freezing process harder and necessitate more sophisticated scheduling schemes to pipeline batches.
- **Limited Streaming Scenarios:** The system deals with generating tokens (streaming output), but is not optimized in a multi-user serving (such as vLLM)

situation, where many inference requests are interleaved in different routing patterns. Load balancing between parallel requests might impair the performance of the cache.

5.3.3 Cache Saturation: Cache Hit Rate

Even when relaxed capacity constraints $C = 8$, $C = 16$ are used, empirical cache hits reach a plateau at a 35 percent level. This ceiling reflects:

- **Routing Distribution Properties:** The expert routing distribution of the MoE models identifies a basic hit rate floor. It has been analyzed that about 35 percent of the logins by experts are always to the subset of experts that is hot and about 65 percent to those experts that are cold and are not resident at the time of access. This distribution does not depend on cache capacity itself and cannot be made better by simply architecturally changing.
- **Limited Temporal Locality:** Tokens can exhibit some clustering (consecutive tokens are likely to route to similar experts) but the routing entropy of individual tokens is large enough that even optimally-sized caches can only obtain a relatively low hit rate.

5.3.4 Staging Buffer Minimalism and Saturation Effects

The empirical finding that the number of stages added to buffers has negligible effect (within a range of $\pm 1.2\%$ deviation) indicates that:

- **Cur Bottleneck:** The bottlenecks at the present are disk I/O and CPU-GPU syncs and not pin memory availability. More staging buffers will do nothing, as long as disk bandwidth or PCIe bandwidth is saturated.
- **Limited Parallelism Gain:** The system is only providing partial overlap of disk I/O, PCIe transfer and computation. The process would demand more complex scheduling e.g. pre-loading the next expert whilst the current expert calculates (speculative loading).

5.3.5 Scope Limitations of Evaluation

The test was thorough but tested on a set of hardware and models:

- **Single Hardware Generation:** NVIDIA T4 and H100 GPUs were experimented with. Performance on AMD ROCm systems, Intel Arc, or older NVIDIA systems (V100, A100) can vary markedly because of varied PCIe bandwidth and memory subsystem attributes and driver optimization.

- **Varying Display of the Model:** Main analysis was performed on Granite-3B. Although the qwen3-MoE and JetMoE are analyzed theoretically, there was no empirical validation of either of the larger models.

5.4 Future Work

In the context of CacheFlow, the basis which has been formed in effect provides a few fruitful research directions:

5.4.1 Embedding-based Predictive pre-fetching

The existing pre-fetching is a reactive type - pre-fetching happens only when a cache miss has been detected. It would predict using:

- **Embedding Analysis:** Determine the semantic relationship between the current token and a token and the upcoming tokens using it to determine which experts get requested in the next few time steps.
- **Speculative Loading:** pre-empt other experts and launch disk reads (or perhaps pre-empt prediction) of the expertise that is more likely to succeed.
- **Adaptive Thresholds:** Modulate the aggressiveness of pre-fetching according to prediction confidence and possible disk bandwidth.

5.4.2 Adaptive and Dynamic Staging Buffer Sizing

Existing staging buffer allocation is turned off. Dynamic sizing would:

- **Monitor I/O Characteristics:** Keep track of disk I/O latency, PCIe transfer throughput, graphics computation time to dynamically allocate staging slots according to bottleneck.
- **Reduce Memory overload:** When disk bandwidth is the bottleneck (as it is now), decrease staging buffers to needy pinned memory. Where the bottleneck of computation is the GPU, expand staging buffers, allowing more aggressive pre-fetching.

5.4.3 Multi-GPU Coherency of Cache and Expert Parallelism

Expansion: To set CacheFlow up in multi-GPU systems, one will need:

- **Distributed Cache State:** Use a coherency scheme (e.g. invalidation-based or write-through) to avoid unnecessary expert fetches in multiple GPUs.

- **Expert Parallelism Scheduling:** When a model includes more than a single disjoint expert group, assign experts to multiple GPUs and cross-gpu communication is used to route tokens and aggregation points.

5.4.4 Token Reordering and Batching by routing

The future batching services in order of arrival. Reordering could:

- **Expert-Aligned Batching:** Recombine tokens in a batch so that tokens which use the same experts are scheduled sequentially, so as to maximize hits in a cache and reduce the number of context switches.
- **Deadlock Avoidance:** Develop design schedules that avoid expert scheduling deadlocks and all cold experts being required to be evicted.

5.4.5 Hardware-Specific Optimizations

- **Direct Disk-to-GPU Transfers:** Consider using GPU direct-attached NVMe (e.g. through NVLink or PCIe peer-to-peer) to avoid using host CPU, halving latency (approximately) to less than 50 ms per expert.
- **Lossless Compression:** On-the-fly decompression of expert weights with the goal of making data both smaller on disk and faster to transfer.
- **Custom Kernels:** Create GPU kernels to be used in highly skilled loading and cache management of devices to minimize the overhead of CPU-GPU synchronization.

5.5 Concluding Remarks

This thesis has shown that Mixture-of-Experts models that have so far only existed on paper and can be studied at best by well funded research institutions can nevertheless be rendered into real, deployable systems by intelligent systems-level optimization to a systems accessible to mainstream researchers and practitioners. The memory hierarchy provided by CacheFlow: seamlessly chaining disk-resident experts together with pinned CPU staging to GPU VRAM slots has shown that the limitations of GPU memory can be overcome to allow deploying state of the art sparse architectures.

The results of the empirical work are impressive: a 590× decrease in GPU memory footprint with a small increase in latencies (at the ultra-constrained conditions, 600 ms to relax to 460 ms) and throughputs up to 130 tokens/minute that facilitate realistic inference workloads. More significantly, the accomplishments open up access to 60+

professional models on consumer-level authorization (12 GB VRAM) that once needed specific infrastructure (80+ GB VRAM).

In addition to the particular technical contributions, this piece covers a major democratization gap in the AI scene. Large language models implemented using MoE architectures are the future of AI research, but they have been limited to organizations that have adequate computer resources. Allowing MoE inference using relatively inexpensive hardware at no cost to the accuracy of computations or ultimate performance means that CacheFlow increases the community of researchers and developers capable of experimenting with, deploying and developing these powerful models.

The constraints mentioned are bottlenecks in the I/O bandwidth, the single-GPU range and the routing distribution limitations, not the technical limitations of the method. They indicate where future research efforts will be needed: predictive prefetching, dynamically scaling the size of buffers, coherency in multi-GPU inference and hardware-enhanced optimizations that can further bridge the gap between a constrained and unlimited-memory inference.

Finally, CacheFlow represents a paradigm of the deployment of sparse neural structures. The system unlocks potential scalability, efficient and accessible AI inference by disaggregating the requirements of model size and the demands of the GPU memory, through clever scheduling and caching. With MoE architecture smattering the literature as well as industry applications, the optimization of systems at the system level such as CacheFlow will be a crucial infrastructure component so that the advantages of these architectures can be achieved not just in literature but also in practice by a wide variety of deployments in the world research and practitioner community.

References

[1] R. A. Jacobs, M. I. Jordan, S. J. Nowlan, and G. E. Hinton, "Adaptive mixtures of local experts," *Neural Computation*, 1991.

[2] M. I. Jordan and R. A. Jacobs, "Hierarchical mixtures of experts and the em algorithm," *Neural Computation*, 1994.

[3] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv preprint arXiv:1412.6980*, 2014.

[4] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," *arXiv preprint arXiv:1503.02531*, 2015.

[5] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," in *International Conference on Learning Representations (ICLR)*, 2015.

[6] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *International Conference on Machine Learning (ICML)*, 2015.

[7] L. J. Ba, J. R. Kiros, and G. E. Hinton, "Layer normalization," *arXiv preprint arXiv:1607.06450*, 2016.

[8] D. Hendrycks and K. Gimpel, "Gaussian error linear units (gelus)," *arXiv preprint arXiv:1606.08415*, 2016.

[9] A. Vaswani, N. Shazeer, N. Parmar, *et al.*, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[10] N. Shazeer, A. Mirhoseini, K. Maziarz, *et al.*, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in *International Conference on Learning Representations (ICLR)*, 2017.

[11] E. Grave, A. Joulin, M. Cissé, D. Grangier, and H. Jégou, "Efficient softmax approximation for gpus," in *International Conference on Machine Learning (ICML)*, 2017.

[12] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, "Improving language understanding by generative pre-training," tech. rep., OpenAI, 2018.

[13] Z. Yang, Z. Dai, R. Salakhutdinov, and W. W. Cohen, "Breaking the softmax bottleneck: A high-rank rnn language model," in *International Conference on Learning Representations (ICLR)*, 2018.

[14] A. Radford, J. Wu, R. Child, *et al.*, "Language models are unsupervised multitask learners," tech. rep., OpenAI, 2019.

[15] R. Child, S. Gray, A. Radford, and I. Sutskever, "Generating long sequences with sparse transformers," in *arXiv preprint arXiv:1904.10509*, 2019.

[16] B. Zhang and R. Sennrich, "Root mean square layer normalization," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[17] T. B. Brown, B. Mann, N. Ryder, *et al.*, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[18] C. Raffel, N. Shazeer, A. Roberts, *et al.*, "Exploring the limits of transfer learning with a unified text-to-text transformer," *Journal of Machine Learning Research (JMLR)*, 2020.

[19] J. Rasley, S. Rajbhandari, O. Ruwase, and Y. He, "Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters," in *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD)*, 2020.

[20] A. Dosovitskiy *et al.*, "An image is worth 16x16 words: Transformers for image recognition at scale," in *International Conference on Learning Representations (ICLR)*, 2020.

[21] N. Shazeer, "Glu variants improve transformer," *arXiv preprint arXiv:2002.05202*, 2020.

[22] I. Beltagy, M. E. Peters, and A. Cohan, "Longformer: The long-document transformer," *arXiv preprint arXiv:2004.05150*, 2020.

[23] Z. Lan, M. Chen, S. Goodman, K. Gimpel, P. Sharma, and R. Soricut, "Albert: A lite bert for self-supervised learning of language representations," in *International Conference on Learning Representations (ICLR)*, 2020.

[24] D. Lepikhin, H. Lee, Y. Xu, *et al.*, "Gshard: Scaling giant models with conditional computation and automatic sharding," in *International Conference on Learning Representations (ICLR)*, 2021.

[25] W. Fedus, B. Zoph, and N. Shazeer, "Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity," *Journal of Machine Learning Research (JMLR)*, 2022.

[26] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "Flashattention: Fast and memory-efficient exact attention with io-awareness," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[27] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with pagedattention," in *ACM Symposium on Operating Systems Principles (SOSP)*, 2023.

[28] H. Touvron, T. Lavril, G. Izacard, *et al.*, "Llama: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.

[29] H. Yoo and H. Lee, "Swapmoe: Serving off-the-shelf moe-based large language models with tunable memory budget," *arXiv preprint arXiv:2309.09515*, 2023.

[30] S. Zuo *et al.*, "Moe-infinity: Efficient moe inference on personal machines with sparsity-aware expert cache," *arXiv preprint arXiv:2305.11794*, 2023.

[31] Microsoft, "Deepspeed-mii: Model inference interface," 2023.

[32] NVIDIA, "Tensorrt-llm: Optimized inference for large language models," 2023.

[33] T. Dao, "Flashattention-2: Faster attention with better parallelism and work partitioning," *arXiv preprint arXiv:2307.08691*, 2023.

[34] W. Liang *et al.*, "Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model," *arXiv preprint arXiv:2405.04434*, 2024.

[35] X. Shen *et al.*, "fmoe: Fine-grained and prompt-aware expert offloading for large mixture-of-experts serving," *arXiv preprint arXiv:2403.12881*, 2024.